Securing your Future with your own Hands

CP08
SEMESTER III

Data Structures With C

SME : Liju Gangadharan

Prepared by:Liju Gangadharan
Modified by: Nayana K
Verified by:Program cooridinator
Approved by: Program cooridinator
Rev No : 01
Released Date: 26-08-2021

Table of Content

1.0	**Introduction**	**2**
1.1	Functions and various implementation of functions Passing values, array, structure, pointer to a function	3
1.2	Structures , structure variables, pointer to structure variables	5
1.3	Pointers, Double pointer, NULL Pointer, void pointer	12
1.4	Dynamic memory allocation	17
2.0	**Stacks**	
2.1	The Stack introduction, Definition and operations on stack. Insert/push operation, delete and pop operation and display	21
2.2	Representation of stacks in C using arrays implementing various operation of stack using C	22
2.3	Stack application: Conversion of expressions with implementation using C	24
2.3.1	Conversion from infix to postfix	24
2.3.2	Conversion from infix to prefix	24
2.3.3	Evaluation of postfix expression	25
2.3.4	Evaluation of prefix expression	26
3.0	**Queues**	
3.1	Introduction, operation and application of queues	27
3.2	Different types of queues - Sequential representation of queues	29
3.2.1	Ordinary queue	30
3.2.2	Circular Queue	30
3.2.3	Priority queue	34
3.2.4	Double Ended queue	35
4.0	**Linked Lists**	
4.1	Introduction	35
4.2	Singly linked list with operations in lists	39
4.3	Creating ordered list, merging two ordered list	39
4.4	Circular single linked list	41
4.5	Circular single linked list with header node	41
4.6	Doubly linked list and Circular double linked list	41
4.7	**Application of linked lists**	47
4.7.1	Addition of two long integers	50
4.7.2	Evaluation of a polynomial	53
4.7.3	Addition of two polynomials	55
4.8	Advantage and disadvantage of different lists	56
5.0	**Trees**	
5.1	Introduction, Definition, storage representation of a binary tree, Terminologies associated with trees	56
5.2	Operations on binary tree Insertion Traversal Searching Copying a Tree	63
5.3	Binary search Tree Insertion Searching Deletion	67

MASTER COPY

5.4 Application of Trees
Searching 67
Sorting
Conversion of Expression
Creating binary tree for postfix expression (Expression Tree)
6.0 Sorting
6.1 Introduction, Bubble Sort 77
6.2 Merge sort 78
6.3 Radix sort 81
6.4 Quick sort 83
6.5 Insertion Sorts
Simple insertion sorts 83
Shell sort
6.6 Heap sort, Priority Queue using heap 86
7.0 Searching 88
7.1 Basic search techniques, algorithmic notation 88
7.2 Searching, searching an ordered Table 89
7.3 Binary search, Interpolation search 90
7.4 Tree searching: Binary search

MASTER COPY

1. INTRODUCTION TO DATA STRUCTURES

Data Structures

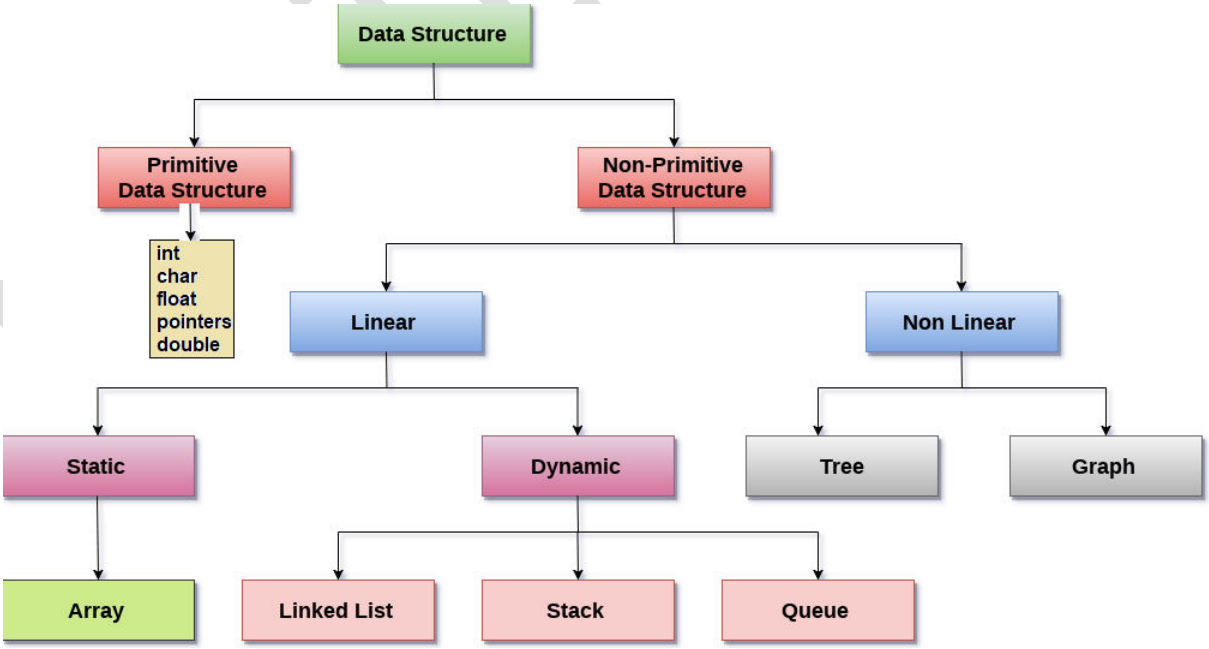
Data structure is a data organization, management, and storage format that enables efficient access and modification. More precisely, a data structure is a collection of data values, the relationships among them, and the functions or operations that can be applied to the data. It is an algebraic structure about data.

Let us consider a training program, Computer Engineering. Student belong to different semesters. Each semester has multiple sections. Each section has students – girls and boys. Each student is described using parameters or attributes such as name, age, marks in various subjects, grade etc. The training program has subjects semester wise, each subject has trainers. There are questions, answers, exam results which are maintained subject wise. There is a relation between student and the subject.

All these entities involve adding, deleting, searching, analysing, modifying of data at every level. How easy are these activities depends on how the data is structured and organized. It also depends on how the relationship between these entities is defined in computer.

Data structure is a method of storing data in a computer, so that it can be used efficiently. Data structure is a study of how the data is stored and organized in computer memory. It mainly deals with:

- **How efficiently the data can be stored in computer memory.**
- **How efficiently data can be retrieved and manipulated.**
- **The possible ways in which different data items are logically related.**



Primitive Data Structures

Primitive data structures are the data structures that can be manipulated directly by machine instructions. Primitive data structure is the basic or

fundamental data types of any programming language.

For example, integers, floating point numbers (real), characters, pointers etc. In C language int, float, char, double are basic data types.

Non Primitive Data Structures

Non primitive data structures are the data structures that cannot be manipulated directly by machine instructions. Arrays, stacks, queues, linked lists, files etc. cannot be manipulated by direct machine instructions. These are generally classified under non-primitive data types. Non primitive data structures classified into two categories, Linear and Non Linear.

a. **Linear data structures** – The data structures which show the relationship of logical adjacency between the data items is stored and retrieved.
   (Arrays, Stacks, Queues, Linked Lists, files)
b. **Non Linear data structures** – The data structures which show the relationship of logical adjacency between the data items is stored and retrieved.
   (Trees, Graphs)

1.1 Functions, Implementations of Functions

A function is a group of statements that together perform a task. Every C program has at least one function, which is **main ()**, and all the most trivial programs can define additional functions.

You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task.

A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

Defining a Function

The general form of a function definition in C programming language is as follows

```
return_type function_name( parameter list )
{
  body of the function
}
```

A function definition in C programming consists of a *function header* and a *function body*. Here are all the parts of a function –

- **Return Type**– A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired

MASTER COPY

operations without returning a value. In this case, the return_type is the keyword **void**.

- **Function Name**– This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body**– The function body contains a collection of statements that define what the function does.

Example:

Given below is the source code for a function called **max()**. This function takes two parameters num1 and num2 and returns the maximum value between the two:

```
/* function returning the max between two numbers */
int max(int num1,int num2)
{
        int result;   /* local variable declaration */
        if(num1 > num2)
            result = num1;
        else
            result = num2;

        return result;
}
```

Function Declarations

A function is declared before it is defined.

A function **declaration** tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately.

A function declaration has the following parts –

```
return_type function_name (parameter list );
```

For the above defined function max(), the function declaration is as follows –

```
int max (int num1, int num2);
```

MASTER COPY

The parameter names are not important in function declaration only their type is required, so the following is also a valid declaration –

```
int max(int, int);
```

Function declaration is required as **extern** when you define a function in one source file and you call that function in another file. In such case, you should declare the function at the top of the file calling the function.

Calling a Function

While creating a C function, you give a definition of what the function has to do. To use a function, you will have to call that function to perform the defined task.

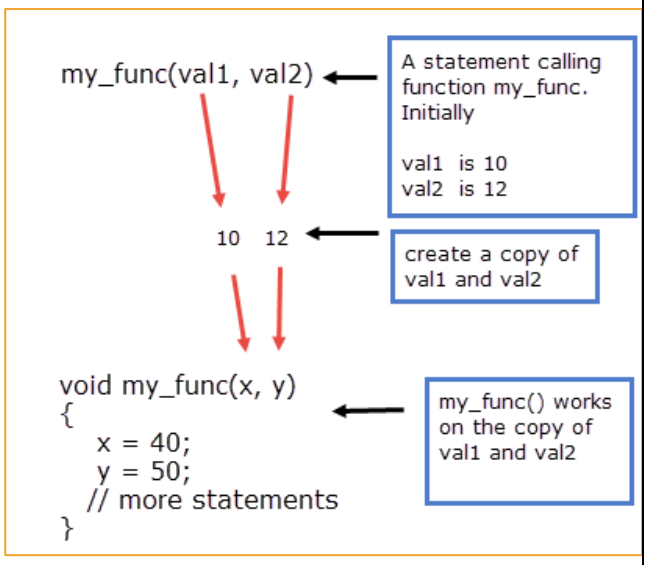
When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

Function Arguments

If a function uses arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.

Formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways in which arguments can be passed to a function:



Call by value: This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.

Call by reference: This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

By default, C uses **call by value** to pass arguments. In general, it means the code within a function cannot alter the arguments used to call the function.

MASTER COPY

1.2 Structures, Structure Variables, Pointer to Structure Variables

Arrays allow to define type of variables that can hold several data items of the same kind. Similarly **structure** is another user defined data type available in C that allows to combine data items of different kinds.

Structures are used to represent a record. Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book –

- Title
- Author
- Subject
- Book ID

Defining a Structure

To define a structure, you must use the **struct** statement. The struct statement defines a new data type, with more than one member. The format of the struct statement is as follows:

```
struct[structure tag]
{
        member definition;
        member definition;
      ...
        member definition;
}[one or more structure variables];
```

The **structure tag** is optional and each member definition is a normal variable definition, such as int i; or float f; or any other valid variable definition. At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is optional. Here is the way you would declare the Book structure:

```
struct Books
{
      char  title[50];
      char  author[50];
      char  subject[100];
      int   book_id;
} book;
```

MASTER COPY

Accessing Structure Members

To access any member of a structure, we use the **member access operator (.)**. The member access operator is coded as a period between the structure variable name and the structure member that we wish to access. You would use the keyword **struct** to define variables of structure type. The following example shows how to use a structure in a program:

```
#include<stdio.h>
#include<string.h>
struct Books
{
        char  title[50];
        char  author[50];
        char  subject[100];
        int   book_id;
};

int main()
{
        struct Books Book1;        /* Declare Book1 of type Book */
        structBooksBook2;          /* Declare Book2 of type Book */

                /* book 1 specification */
        strcpy(Book1.title,"C Programming");
        strcpy(Book1.author,"Nuha Ali");
        strcpy(Book1.subject,"C Programming Tutorial");
        Book1.book_id = 6495407;

                /* book 2 specification */
        strcpy(Book2.title,"Telecom Billing");
        strcpy(Book2.author,"Zara Ali");
        strcpy(Book2.subject,"Telecom Billing Tutorial");
        Book2.book_id = 6495700;

                /* print Book1 info */
          printf("Book 1 title : %s\n",Book1.title);
          printf("Book 1 author : %s\n",Book1.author);
          printf("Book 1 subject : %s\n",Book1.subject);
          printf("Book 1 book_id : %d\n",Book1.book_id);

                /* print Book2 info */
```

MASTER COPY

```
        printf("Book 2 title : %s\n",Book2.title);
        printf("Book 2 author : %s\n",Book2.author);
        printf("Book 2 subject : %s\n",Book2.subject);
        printf("Book 2 book_id : %d\n",Book2.book_id);

    return0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Book 1 title : C Programming
Book 1 author : Nuha Ali
Book 1 subject : C Programming Tutorial
Book 1 book_id : 6495407
Book 2 title : Telecom Billing
Book 2 author : Zara Ali
Book 2 subject : Telecom Billing Tutorial
Book 2 book_id : 6495700
```

Structures as Function Arguments

You can pass a structure as a function argument in the same way as you pass any other variable or pointer.

```
#include<stdio.h>
#include<string.h>
struct Books
{
    char  title[50];
    char  author[50];
    char  subject[100];
    int   book_id;
};

    /* function declaration */
    void printBook(struct Books book );

int main()
{
    struct Books Book1;       /* Declare Book1 of type Book */
    structBooksBook2;         /* Declare Book2 of type Book */

        /* book 1 specification */
```

MASTER COPY

```
    strcpy(Book1.title,"C Programming");
    strcpy(Book1.author,"Nuha Ali");
    strcpy(Book1.subject,"C Programming Tutorial");
    Book1.book_id = 6495407;

        /* book 2 specification */
    strcpy(Book2.title,"Telecom Billing");
    strcpy(Book2.author,"Zara Ali");
    strcpy(Book2.subject,"Telecom Billing Tutorial");
    Book2.book_id = 6495700;

    printBook(Book1);     /* print Book1 info */
    printBook(Book2);     /* print Book2 info */

    return0;
}

void printBook(struct Books book )
{
        printf("Book title : %s\n", book.title);
        printf("Book author : %s\n", book.author);
        printf("Book subject : %s\n", book.subject);
        printf("Book book_id : %d\n", book.book_id);
}
```

When the above code is compiled and executed, it produces the following result:
Book title : C Programming
Book author : Nuha Ali
Book subject : C Programming Tutorial
Book book_id : 6495407
Book title : Telecom Billing
Book author : Zara Ali
Book subject : Telecom Billing Tutorial
Book book_id : 6495700

Pointers to Structures

You can define pointers to structures in the same way as you define pointer to any other variable:

```
struct Books *struct_pointer;
```

MASTER COPY

Now, you can store the address of a structure variable in the above defined pointer variable. To find the address of a structure variable, place the '&'; operator before the structure's name as follows −

```
struct_pointer = &Book1;
```

To access the members of a structure using a pointer to that structure, you must use the → operator as follows −

```
struct_pointer->title;
```

Let us re-write the above example using structure pointer.

```
#include<stdio.h>
#include<string.h>
struct Books
{
        char  title[50];
        char  author[50];
        char  subject[100];
        int   book_id;
};

        /* function declaration */
        void printBook(struct Books book );

int main()
{
        struct Books Book1;       /* Declare Book1 of type Book */
        structBooksBook2;         /* Declare Book2 of type Book */

                /* book 1 specification */
        strcpy(Book1.title,"C Programming");
        strcpy(Book1.author,"Nuha Ali");
        strcpy(Book1.subject,"C Programming Tutorial");
        Book1.book_id = 6495407;

                /* book 2 specification */
        strcpy(Book2.title,"Telecom Billing");
        strcpy(Book2.author,"Zara Ali");
        strcpy(Book2.subject,"Telecom Billing Tutorial");
        Book2.book_id = 6495700;
```

MASTER COPY

```
        printBook(&Book1);   /* print Book1 info by passing address of Book1 */
        printBook(&Book2);   /* print Book2 info by passing address of Book2 */

        return0;
}

void printBook(struct Books *book )
{
          printf("Book title : %s\n", book->title);
          printf("Book author : %s\n", book->author);
          printf("Book subject : %s\n", book->subject);
          printf("Book book_id : %d\n", book->book_id);
}
```

When the above code is compiled and executed, it produces the following result –

Book title : C Programming
Book author : Nuha Ali
Book subject : C Programming Tutorial
Book book_id : 6495407
Book title : Telecom Billing
Book author : Zara Ali
Book subject : Telecom Billing Tutorial
Book book_id : 6495700

1.3 Pointers, Double Pointer, NULL Pointer, void Pointer

Pointers in C are easy and fun to learn. Some C programming tasks are performed more easily with pointers, and other tasks, such as dynamic memory allocation, cannot be performed without using pointers. So it becomes necessary to learn pointers to become a perfect C programmer. Let's start learning them in simple and easy steps.

As you know, every variable is a memory location and every memory location has its address defined which can be accessed using ampersand (&) operator, which denotes an address in memory. Consider the following example, which prints the address of the variables defined –

```
#include<stdio.h>

int main ()
```

MASTER COPY

```
{

        int  var1;
        char var2[10];

          printf("Address of var1 variable: %x\n", &var1  );
          printf("Address of var2 variable: %x\n", &var2  );

        return0;
}
```

When the above code is compiled and executed, it produces the following result:

Address of var1 variable: bff5a400
Address of var2 variable: bff5a3f6

Pointers

A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address. The general form of a pointer variable declaration is:

```
type *var-name;
```

Here, **type** is the pointer's base type; it must be a valid C data type and **var-name** is the name of the pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer. Take a look at some of the valid pointer declarations:

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

Use of Pointers

There are a few important operations, which we will do with the help of pointers very frequently.

MASTER COPY

a. We define a pointer variable.
b. Assign the address of a variable to a pointe.r
c. Access the value at the address available in the pointer variable. This is done by using unary operator * that returns the value of the variable located at the address specified by its operand.

The following example makes use of these operations –

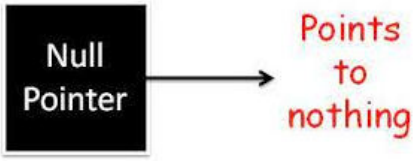
```
#include<stdio.h>
int main ()
{
        int var = 20; /* actual variable declaration */
        int *ip;        /* pointer variable declaration */
        ip = &var;     /* store address of var in pointer variable*/

        printf("Address of var variable: %x\n",&var);

                /* address stored in pointer variable */
        printf("Address stored in ip variable: %x\n", ip );

                /* access the value using the pointer */
        printf("Value of *ip variable: %d\n",*ip );

        return0;
}
```

When the above code is compiled and executed, it produces the following result –

Address of var variable: bffd8b3c
Address stored in ip variable: bffd8b3c
Value of *ip variable: 20

NULL Pointers

It is always a good practice to assign a NULL value to a pointer variable in case you do not have an exact address to be assigned. This is done at the time of variable declaration. A pointer that is assigned NULL is called a **null** pointer.



The NULL pointer is a constant with a value of zero defined in several standard libraries. Consider the following program –

```
#include<stdio.h>
int main ()
```

MASTER COPY

```
{
        Int *ptr = NULL;
        printf("The value of ptr is : %x\n", ptr  );
        return0;
}
```

When the above code is compiled and executed, it produces the following result –

The value of ptr is 0

In most of the operating systems, programs are not permitted to access memory at address 0 because that memory is reserved by the operating system. However, the memory address 0 has special significance; it signals that the pointer is not intended to point to an accessible memory location. But by convention, if a pointer contains the null (zero) value, it is assumed to point to nothing.

To check for a null pointer, you can use an 'if' statement as follows:

```
if(ptr)    /* succeeds if p is not null */
if(!ptr)   /* succeeds if p is null */
```

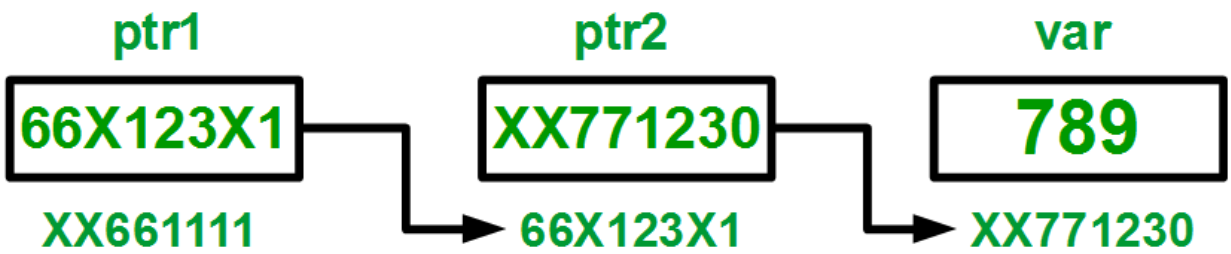
Double Pointer (Pointer to Pointer) in C

We already know that a pointer points to a location in memory and thus used to store address of variables. So, when we define a pointer to pointer. The first pointer is used to store the address of second pointer. That is why they are also known as double pointers.

Declaring Pointer to Pointer is similar to declaring pointer in C. The difference is we have to place an additional '*' before the name of pointer.

```
int **ptr;    // declaring double pointers
```

Below diagram explains the concept of Double Pointers:



The above diagram shows the memory representation of a pointer to

MASTER COPY

pointer. The first pointer ptr1 stores the address of the second pointer ptr2 and the second pointer ptr2 stores the address of the variable.
Let us understand this more clearly with the help of below program:

```
int main()
{
        int var = 789;
        int *ptr2;     // pointer for var
        int **ptr1;   // double pointer for ptr2

        ptr2 = &var;   // storing address of var in ptr2
        ptr1 = &ptr2;     // Storing address of ptr2 in ptr1

        // Displaying value of var using both single and double pointers
        printf("Value of var = %d\n", var );
        printf("Value of var using single pointer = %d\n", *ptr2 );
        printf("Value of var using double pointer = %d\n", **ptr1);

       return 0;
}
```

Output:
Value of var = 789
Value of var using single pointer = 789
Value of var using double pointer = 789

void pointer in C

A void pointer is a pointer that has no associated data type with it. A void pointer can hold address of any type and can be typcasted to any type.

```
int a = 10;
char b = 'x';

void*p = &a;  // void pointer holds address of int 'a'
p = &b; // void pointer holds address of char 'b'
```

MASTER COPY

When We assign address of integer variable to void pointer	Void Pointer Becomes Integer Pointer
When We assign address of character variable to void pointer	Void Pointer Becomes Character Pointer
When We assign address of floating variable to void pointer	Void Pointer Becomes Floating Pointer

1.4 Dynamic Memory Allocation

In C, the exact size of array is unknown until compile time, i.e., the time when a compiler compiles your code into a computer understandable language. So, sometimes the size of the array can be insufficient or more than required.

Dynamic memory allocation allows your program to obtain more memory space while running, or to release it if it's not required.

In simple terms, Dynamic memory allocation allows you to manually handle memory space for your program.

malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	deallocate the previously allocated space
realloc()	Change the size of previously allocated space

malloc()

The name malloc stands for "memory allocation".

The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.

Syntax of malloc()

```
ptr = (cast-type*) malloc(byte-size)
```

Here, *ptr* is pointer of cast-type. The malloc() function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns NULL pointer.

```
ptr = (int*) malloc(100 * sizeof(int));
```

```
#include<stdio.h>
#include<stdlib.h>

int main()
```

MASTER COPY

```
{
        int num, i,*ptr, sum =0;
        printf("Enter number of elements: ");
        scanf("%d",&num);

        ptr = (int*) malloc(num *sizeof(int));  //memory allocated using malloc
        if(ptr == NULL)
        {
                printf("Error! memory not allocated.");
                exit(0);
        }

        printf("Enter elements of array: ");
        for(i =0; i < num;++i)
        {
            scanf("%d", ptr + i);
            sum +=*(ptr + i);
        }
        printf("Sum = %d", sum);
        free(ptr);
        return 0;
}
```

calloc()

The name calloc stands for "contiguous allocation".

The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size and sets all bytes to zero.

Syntax of calloc()

```
ptr = (cast-type*)calloc(n, element-size);
```

This statement will allocate contiguous space in memory for an array of *n* elements.

For example:

```
ptr = (float*) calloc(25, sizeof(float));
```

free()

Dynamically allocated memory created with either calloc() or malloc() doesn't get freed on its own. You must explicitly use free() to release the space.

MASTER COPY

syntax of free()

```
free(ptr);
```

This statement frees the space allocated in the memory pointed by ptr.

Example: Write a C program to find sum of n elements entered by user. To perform this program, allocate memory dynamically using calloc() function.

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
	int num, i,*ptr, sum =0;
	printf("Enter number of elements: ");
	scanf("%d",&num);
	ptr =(int*) calloc(num,sizeof(int));
	if(ptr == NULL)
	{
		printf("Error! memory not allocated.");
		exit(0);
	}
	printf("Enter elements of array: ");
	for(i =0; i < num;++i)
	{
		scanf("%d", ptr + i);
		sum +=*(ptr + i);
	}
	printf("Sum = %d", sum);
	free(ptr);
	return0;
}
```

realloc()

If the previously allocated memory is insufficient or more than required, you can change the previously allocated memory size using realloc().

Syntax of realloc()

```
ptr = realloc(ptr, newsize);
```

```
#include<stdio.h>
#include<stdlib.h>
```

MASTER COPY

```
int main()
{
        int*ptr, i , n1, n2;
        printf("Enter size of array: ");
        scanf("%d",&n1);
        ptr =(int*) malloc(n1 *sizeof(int));
        printf("Address of previously allocated memory: ");
        for(i =0; i < n1;++i)
                printf("%u\t",ptr + i);
         printf("\nEnter new size of array: ");
        scanf("%d",&n2);
        ptr = realloc(ptr, n2);
        for(i =0; i < n2;++i)
                printf("%u\t", ptr + i);
        return0;
}
```

MASTER COPY

2. STACKS

2.1 Stack - Push, Delete, Pop and Display

Stack is a linear data structure, where elements can be inserted at one end and the elements are deleted from the same end of the stack. The insertion and deletion happens at one end of the stack. That end is known as top.

In this approach the last inserted element is the first to be deleted out from stack. Hence it is also called is LIFO (Last In First Out). Stack is a data structure which follows LIFO.

A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or a pile of plates, etc.

Real time examples

Stack of chairs

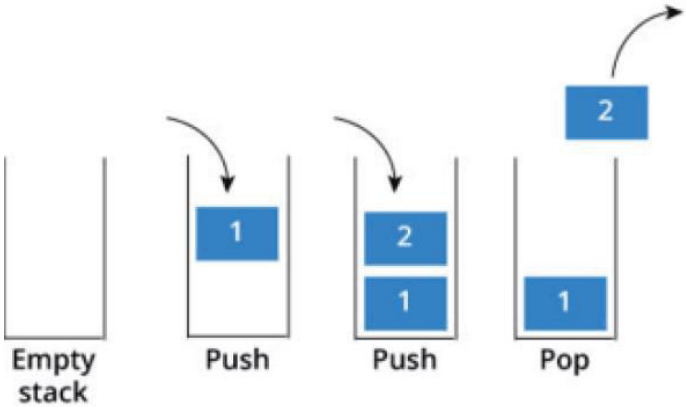
stack of books

stack of coins

stack of plates

A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from the top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.

A stack can be implemented by means of Array, Structure, Pointer, and Linked List. Stack can either be a fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays, which makes it a fixed size stack implementation.



2.2 Implementing stack in C using arrays

Operations on Stack.

a. **Push**
b. **Pop**
c. **Display**

Other operations on stack are:

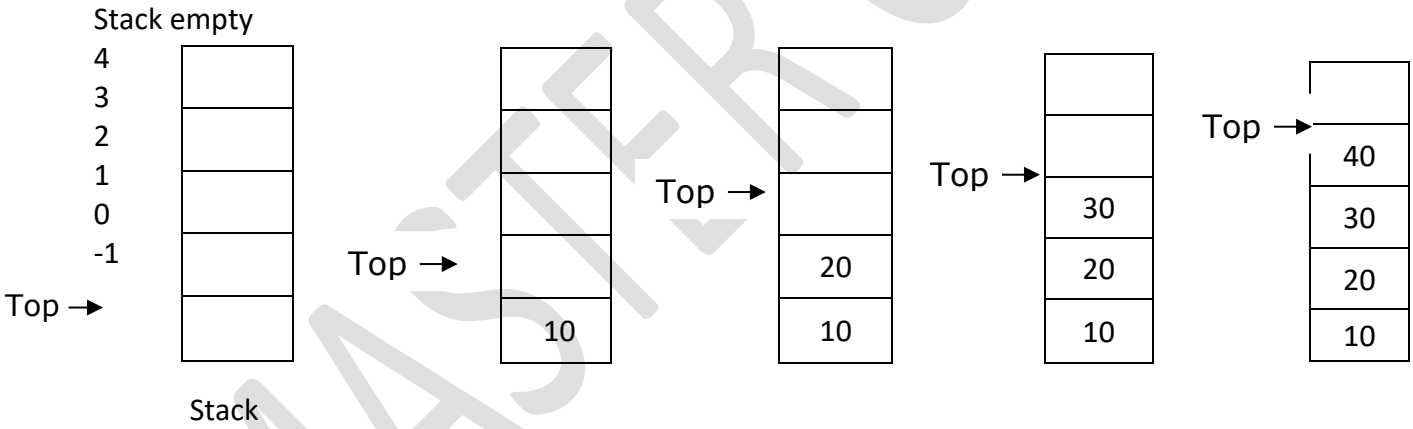
peek()	returns a copy of the element on the top of the stack without removing it
is_empty()	checks whether a stack is empty
is_full()	checks whether a stack is at maximum capacity when stored in a static (fixed-size) structure

a. Push (Insertion)

Push operation inserts or adds one element on the top of a stack. Only one item is inserted during one push operation and the element inserted on the top of the stack using **push().** One push makes one increment on top.

Top = Top + 1

Push operation on stack of maximum size is 5.



As we do push operation, the stack starts to get filled up. There is a possibility of stack getting full. If stack is full then further insertion on stack is not possible. This state of stack is called as **Stack Overflow**.

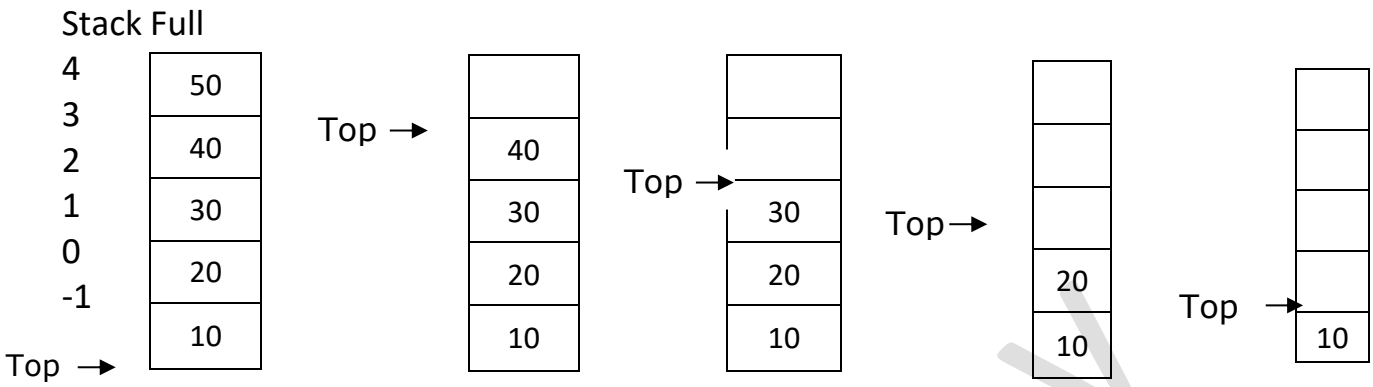
b. Pop (Deletion)

Pop operation retrieves and deletes one element from the top of a stack. Only one item is deleted during one pop operation and the top most element of the stack (last inserted item) will be deleted. One pop operation decrements the value of top by one

Top = Top - 1

MASTER COPY

Pop operation on stack of maximum size is 5



When working with stack, the push and pop operations happen interchangeably depending on the application. Every pop operation deletes one item from the stack, and then there is a possibility of stack become empty with no elements on stack. It is not possible to delete an item from the stack at this point, then trying to delete and item from stack results in **Stack Underflow**.

c. Display

In display, if stack is not empty then display all the items one after the another. If no items are available the appropriate error messages has to display. The display operation uses **pop()** to retrieve and delete the items from top of the stack till the stack is empty.

2.3 Stack Application: Conversion of expressions

Applications of Stack

a. **Conversion of expressions**
b. **Evaluation of expressions**
c. **Recursion**
d. **Other applications**

Mathematical expressions can be represented in various ways.

i. **Infix**
ii. **Prefix**
iii. **Postfix**

a. Conversion of Mathematical Expressions

When we use mathematical expressions in program, often we refer to used infix notations. These expressions will be converted into corresponding equivalent computer understandable expressions using stack. Using stack we can efficiently convert expressions from infix to postfix or infix to prefix notations.

b. Evaluation of Expressions

MASTER COPY

Finding the result of an arithmetic expression is called evaluation of expressions. The expressions written in prefix or postfix can be easily evaluated by using stack.

c. Recursion

A function calling itself is called as a recursive function.

d. Other Applications

There are many other applications where stacks can be used. Example: Checking if string is palindrome, validity of an expression etc

Need of converting expressions to different notations

An arithmetic expressions which is represented in infix notation is easily understandable to human, but it is not so easy for computer to evaluate and give the result. It is required to convert these infix expressions into best understandable format of computers which is postfix or prefix notation.

Infix Notation

Example : (a + b) This is easily understood by humans.

Operators are placed in between operands in infix notation. Here '+' is an operator and 'a' and 'b' are operands .

Example (5+3), (4*3–2), (8–3*2), (5^2)

Prefix Notation

Example: (+ ab)

In prefix notation, operators come first, followed by operands.

Postfix Notation

Example: (ab +)

In postfix notation, operands come first, followed by operator

Infix	Postfix	Prefix
a + b	a b +	+ a b
(a+ (b – c) * d)	a b c – d * +	+a*-bcd
a*b/c-e+d	ab*c/e-d+	+-/*abced
a^b^c+d/e	abc^^de/+	+/^^bcade

2.3.1 Conversion From Infix To Postfix

In normal algebra we use the infix notation like a+b*c. The corresponding postfix notation is abc*+. The algorithm for the conversion is as follows :

Scan the Infix string from left to right. Initialise an empty stack. If the scanned character is an operand, add it to the Postfix string. If the scanned character is an operator and if the stack is empty Push the character tostack.

MASTER COPY

If the scanned character is an Operand and the stack is not empty, compare the precedence of the character with the element on top of the stack (top Stack). If topStack has higher precedence over the scanned character Popthe stack else Push the scanned character to stack. Repeat this step as long as stack is not empty and topStack has precedence over the character.

Repeat this step till all the characters are scanned. (After all the characters are scanned, we have to add any character that the stack may have to the Postfix string). If stack is not empty add top Stack to Postfix string and Pop the stack.

Repeat this step as long as stack is not empty. Return the Postfix string.

2.3.2 Conversion from infix to prefix

Program to convert from Infix to Prefix

```
#include<stdio.h>
#include<conio.h>
void inf_pref(char[],char[]);
int SP(char);
int IP(char);
void main()
{
	char inf[20], pref[20];
	clrscr();
	printf("\nEnter infix expression:");
	scanf("%s",inf);
	inf_pref(inf, pref);
	printf("\nPrefix expression:%s",pref);
	getch();
}
void inf_pref(char inf[],char pref[])
{
	int i, j, top = -1;
	char s[30], sym;
	s[++top] = '#';
	j = 0;
	strrev(inf);
	for(i=0; i<strlen(inf); i++)
	{
		sym=inf[i];
		while(SP(s[top])>IP(sym))
		{
			pref[j]=s[top--];
			j++;
```

MASTER COPY

```
                }
                If (SP(s[top])! = IP(sym))
                        s[++top]=sym;
                else
                        top--;
        }
        while(s[top]!='#')
        {
                pref[j++]=s[top--];
        }
        pref[j]='\0';
        strrev(pref);
}
int SP(char sym)
{
        switch(sym)
        {
                case '+':
                case '-': return 1;
                case '*':
                case '/': return 3;
                case '$':
                case '^': return 6;
                case ')': return 0;
                case '#': return -1;
                default : return 8;
        }
}
int IP(char sym)
{
        switch(sym)
        {
                case '+':
                case '-': return 2;
                case '*':
                case '/': return 4;
                case '$':
                case '^': return 5;
                case '(': return 0;
                case ')': return 9;
                default : return 7;
```

MASTER COPY

```
        }
}
```

2.3.3 Evaluation of Postfix Expression

Algorithm

Step 1: Scan the input symbols from left to right
Step 2: if the symbol is operand the push into the stack
Step 3: if the symbol is an operator, pop two items from the
Stack and assign to operend2 and operand1
Op2= stk[top--];
Op1= stk[top--];
Step 4: Evaluate op1 and op2 with the symbol operator
Step 5: Result of evaluation has to push into stack
Step 6: Continue the process to examine all the symbols in the
input.
Step 7: The end of iteration the evaluated results will be available in the stack.

2.3.4 Evaluation of Prefix Expression

Program to Evaluate Prefix expression

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int evalPrefix(int,int,char);
void main()
{
        char suff[20],stk[20],sym,op1,op2;
        int i,top=-1,res,j;
        clrscr();
        printf("\nEnter a valid Prefix expression:");
        scanf("%s",suff);
        strrev(suff);
        for(i=0;i<strlen(suff);i++)
        {
                sym=suff[i];
                if(isdigit(sym))
                stk[++top]=sym-'0';
                else
                {
                        op1=stk[top--];
                        op2=stk[top--];
                        res=evalPrefix(op1,op2,sym);
```

MASTER COPY

```
                        stk[++top]=res;
                }
        }
        res=stk[top--];
        printf("\nResult:%d",res);
        getch();
}
int evalPrefix(int op1,int op2,char op)
{
        switch(op)
        {
                case '+':op1=op1+op2;
                break;
                case '-':op1=op1-op2;
                break;
                case '*':op1=op1*op2;
                break;
                case '/':op1=op1/op2;
                break;
        }
        return op1;
}
```

MASTER COPY

QUEUES - III

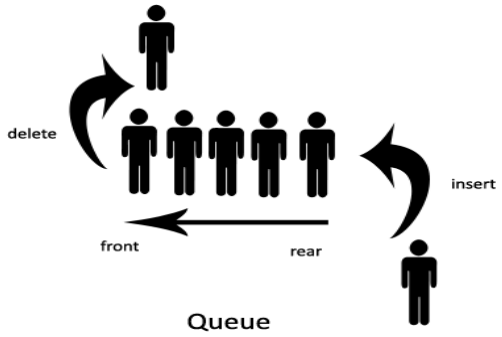
3.1 Introduction, operation and application of queues

Definition

Queue is a linear data structure, where elements can be inserted at one end and the elements are deleted from another end. The insertion happened at one end called rear and deletion happens at another end called front.

In this approach generally the first inserted elements is the first to be deleted out from the queue. Hence it is also called is FIFO (First In First Out). Queue is the data structure which follows FIFO.

Real time examples



Operations on Queues.

1. Qinsert (Enqueue)
2. Qdelete (Dequeue)
3. QDisplay

QInsert (Enqueue)

Adding one element or an item in a queue is called Qinsert also referred as Enqueue. Element will be inserted on rear end of the queue. One Qinsert makes certain changes on queue. Ie

Rear = Rear + 1

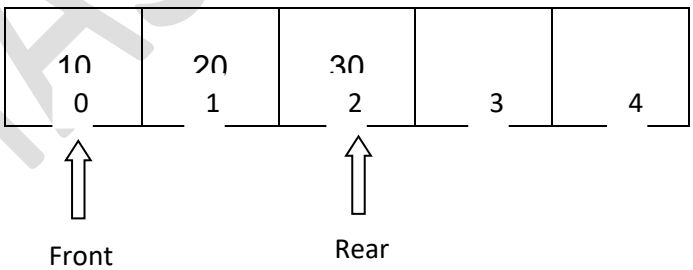


Fig (1)

Fig (1): Insertion in queue of maximum size is five. (3 elements are inserted into this queue(10, 20,30), the rear points to the last inserted item and the front points to the first inserted item.

MASTER COPY

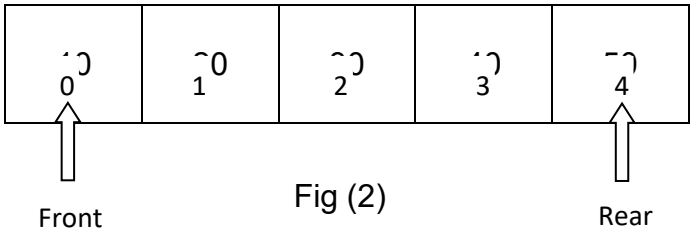


Fig (2)

Fig (2): Queue Full or Queue overflow state. Rear reaches the maximum size then queue is full.

2. QDelete (Dequeue)

Deleting one element from the queue is called as Dequeue. Qdelete will delete or retrieve one element from 'front' of the queue. One queue delete will makes following changes on queue.

Front = Front + 1

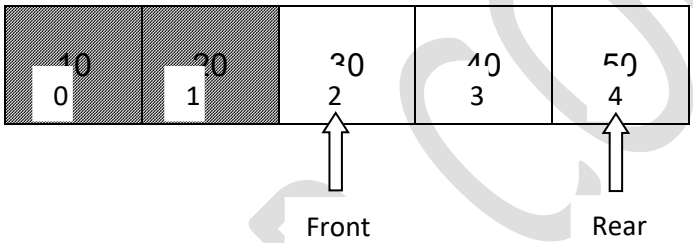


Fig (3)

Fig (3): Deletion from a queue of maximum size is five and inserted five elements into the queue (10, 20, 30, 40, and 50). The item first deleted will be 10 and followed by 20 and so on since it follows FIFO.

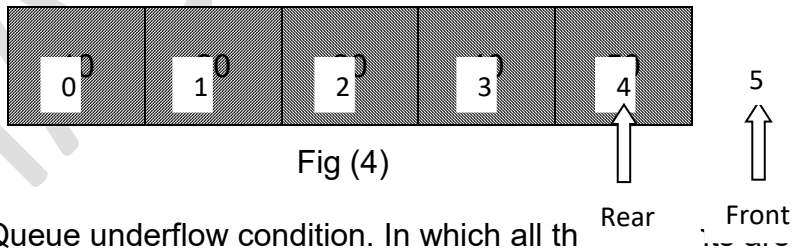
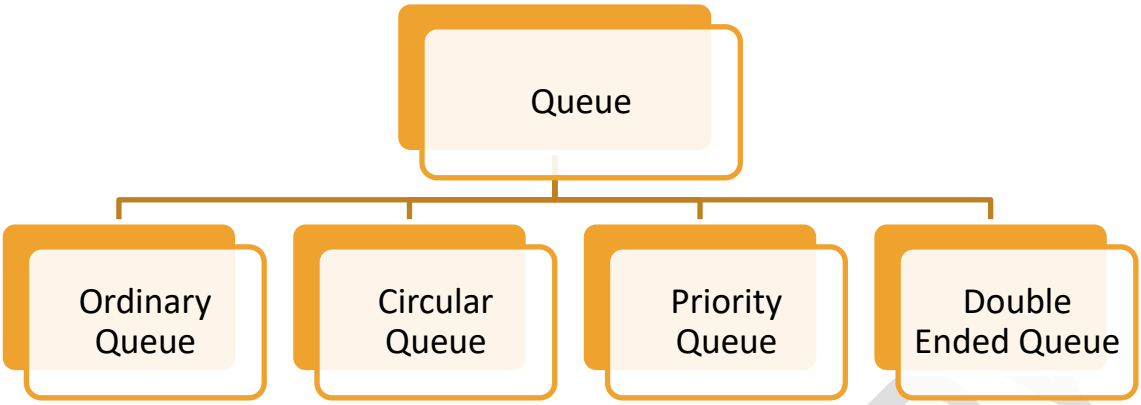


Fig (4)

Fig (4): shows the Queue underflow condition. In which all th [illegible] deleted from the queue and finally 'front' value become greater than 'rear'.

3.2 Different Types of Queues.

MASTER COPY



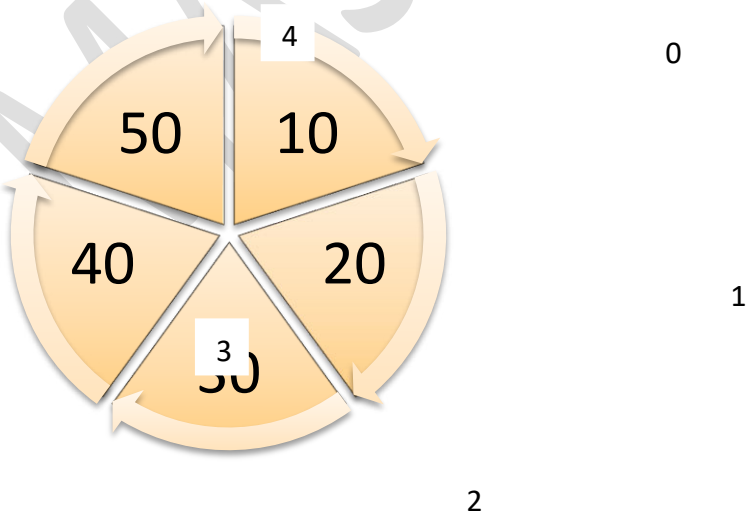
3.2.1 Ordinary Queue

A queue is defined as a special type of data structure where elements are inserted from one end and elements are deleted from another end. The end from where elements are inserted is called as **rear end** and the end from where elements are deleted is called as **front end.** Ordinary queue follows FIFO(Last In First Out). The element first inserted will be deleted or come out of queue first.

Array implementation of Ordinary Queue. (Program)

3.2.2 Circular Queue

In Circular queue elements will be inserted and deleted in a circular fashion. Circular queue follows FIFO (First in First Out).
Pictorial representation of Circular queue



Insertion in Circular Queue

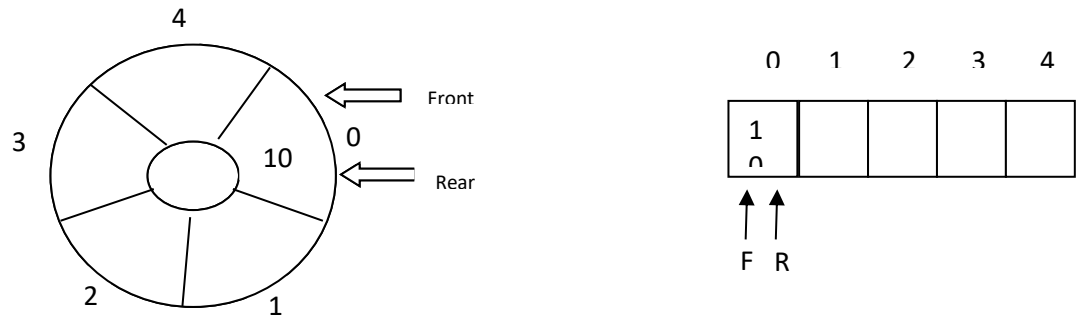


Fig (1–a)

Fig(1-a) Shows the pictorial representation of queue of Maximum size is 5. After inserting one element 10 into the queue. Front and rear points the first array index when inserted first element into queue.

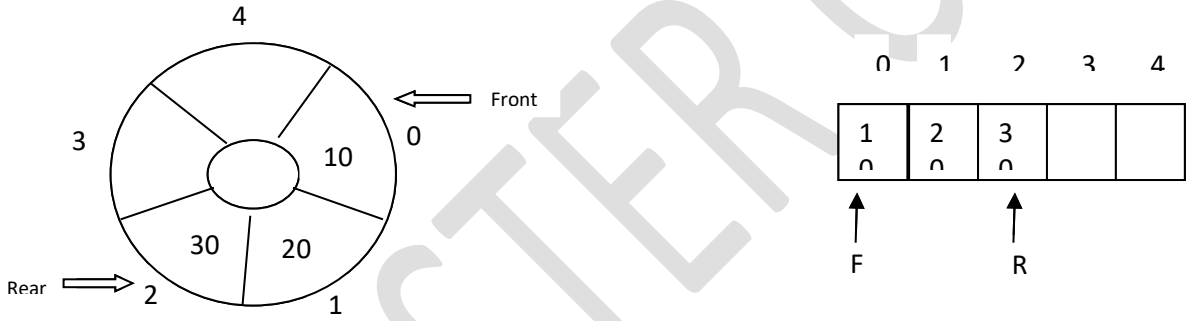


Fig (1–b)

Fig(1-b) Shows inserting three elements into the queue 10, 20 and 30. Front points to the first array index where first item(10) is inserted and rear points to the last inserted item into the queue (30).

MASTER COPY

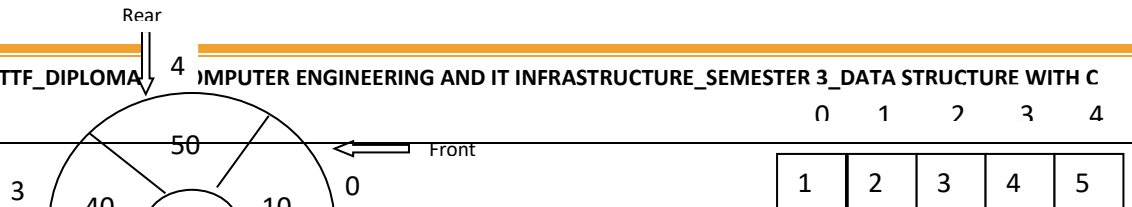


Fig (1–c)

Fig(1-c) Shows circular queue full condition. Inserted five items into queue 10, 20,30, 40 and 50. When queue reaches the maximum size then its called as Queue Overflow or Queue full condition.

Deletion in Circular Queue

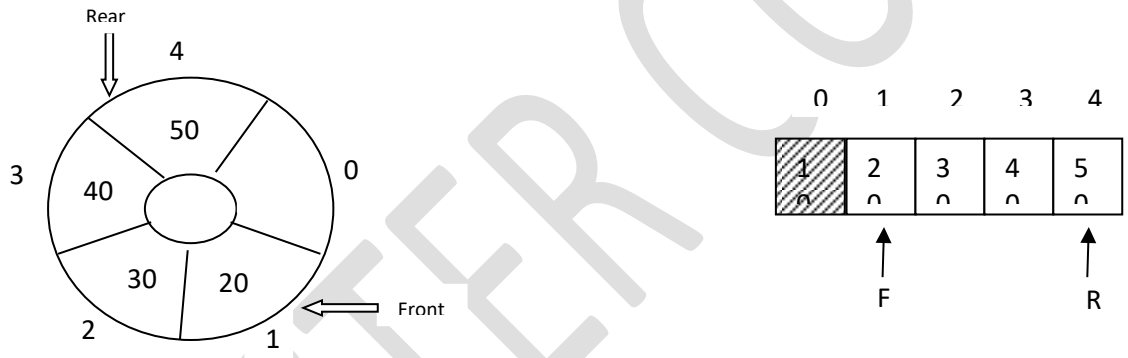
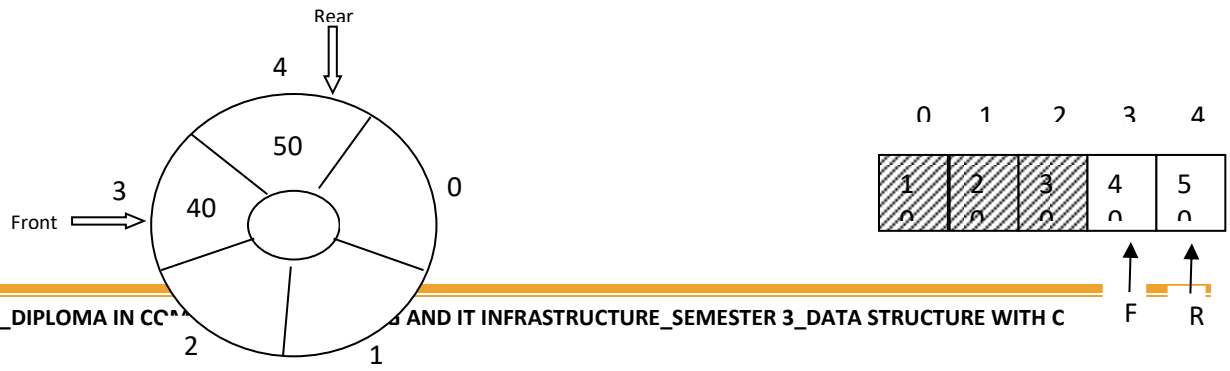


Fig (2–a)

Fig(2-a) From a full queue, the item 10 deleted since it's the first item arrived in the queue. After deletion front point to the index 1 and rear remains same.



MASTER COPY

Fig (2–b)

Fig(2-b) shows two more elements deleted from queue 20 and 30.

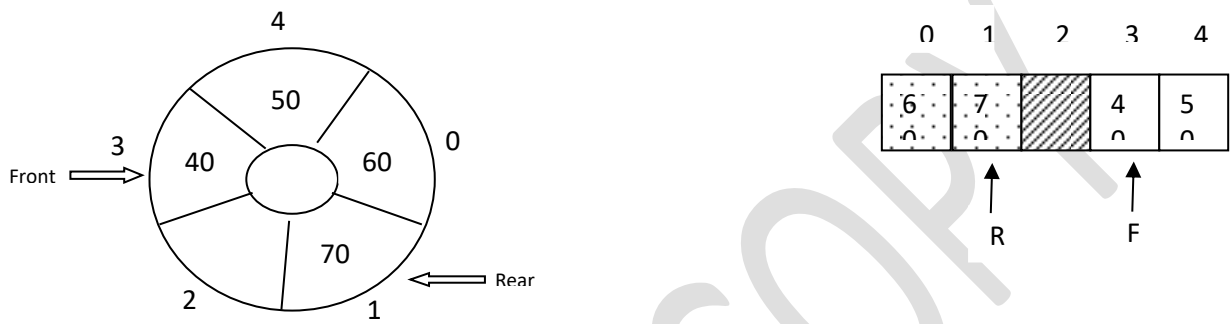


Fig (2–c)

Fig(2-c) Reinsertion of elements 60 and 70 into deleted locations ie index 0 and 1

3.2.3 Priority Queue

Priority queue is a special type of data structure in which items can be inserted or deleted based on the given priority. Always an element with highest priority is processed before processing the lowest priority elements. If the elements in the queue come with same or equal priority, then the element which is inserted first will be processed first.

Ascending Priority Queue.

In ascending priority queue elements can be inserted in any order, while deleting an element from queue the highest priority items will be deleted first ie the smallest element in the queue will be removed first.

Ascending order priority queue treat the smallest number has highest priority.

Descending Priority Queue.

In descending priority queue also elements can be inserted in any order, while deleting an element from queue the highest priority items ie the largest element in the queue will be deleted

Descending order priority queue treat the largest number in the queue with highest priority.

MASTER COPY

3.2.4 Double Ended Queue (Deque)

A deque is a special data structure in which insertions and deletions can be performed at both the ends ie rear and front ends. Operations that can be performed on deque are

1. Insertion on Rear
2. Insertion on Front
3. Deletion from Front
4. Deletion from rear

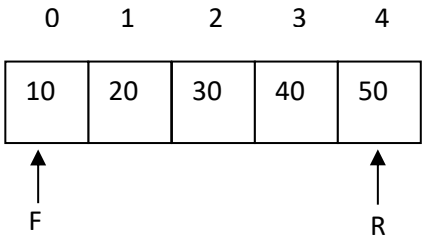


Fig (4-a) shows double ended queue is full. Inserted elements using rear insertion. (rear

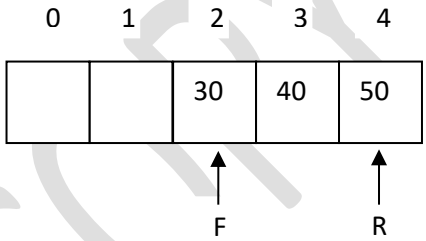


Fig (4-a) shows delete two elements 10, 20 using front delete (Front increment)

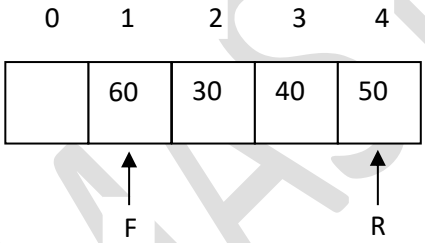


Fig (4-c) shows insertion using front 60 inserted on 1 index. (Front decrement)

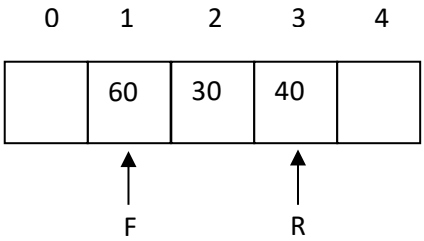
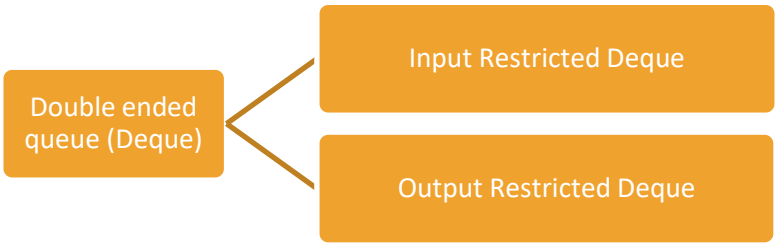


Fig (4-d) shows deletion from rear. The element 50 deleted from the queue using (Rear decrement)

Two variants of Double ended queue



Input Restricted Deque

An Input Restricted Deque is a variation of Double Ended Queue (Deque), which supports following operations

Delete front
Delete rear
Insert front
Display

Unlike Deque it won't support insertion rear operation on Input restricted deque.

Output Restricted Deque

An Output Restricted Deque is a another variation of Double Ended Queue (Deque), which supports following operations

Delete front
Insert rear
Insert front
Display

Unlike Deque it won't support deletion rear operation on Output restricted deque.

MASTER COPY

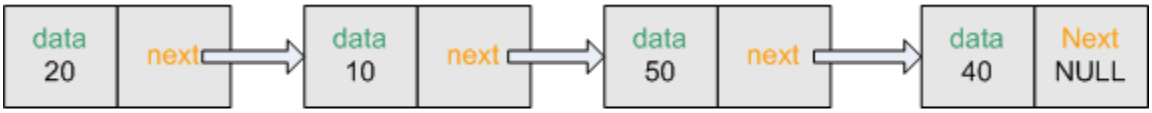
UNIT 4- Linked List

4.1 Introduction

What is Linked List?

In computer science, a **linked list** is a data structure consisting of a group of nodes which together represent a sequence. Under the simplest form, each node is composed of a datum and a reference (in other words, a *link*) to the next node in the sequence; more complex variants add additional links. This structure allows for efficient insertion or removal of elements from any position in the sequence.

Introduction



Linked list

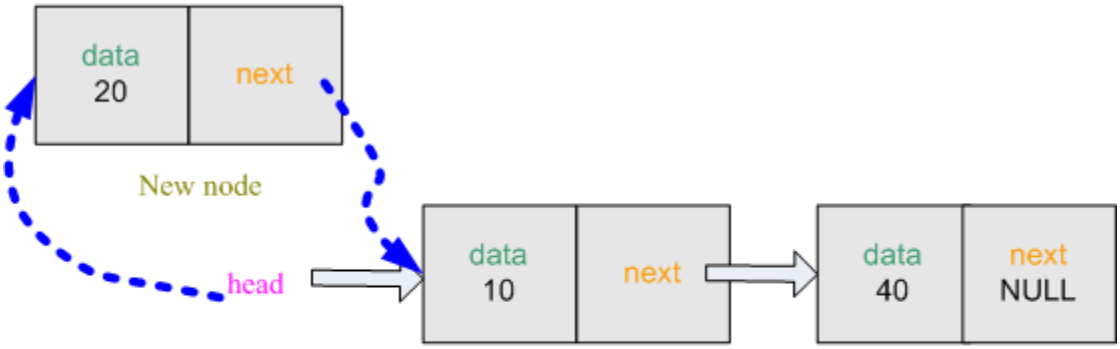
Linked list is one of the fundamental data structures, and can be used to implement other data structures.

In a **linked list** there are different numbers of nodes. Each node is consists of two fields. The first field holds the value or data and the second field holds the reference to the next node or null if the **linked list** is empty.
Pointer

To create **linked list** in C/C++ we must have a clear understanding about pointer. Now I will explain in brief what is pointer and how it works.

A pointer is a variable that contains the address of a variable. The question is why we need pointer? Or why it is so powerful? The answer is they have been part of the C/C++ language and so we have to use it. Using pointer we can pass argument to the functions. Generally we pass them by value as a copy. So we cannot change them. But if we pass argument using pointer, we can modify them. To understand about pointers, we must know how computer store variable and its value. Now, I will show it here in a very simple way.

MASTER COPY



Linked list

Link-List is a **data structure** in computer science terms. It is more of an advanced array or rather a dynamic array. Link-list is a grouping of data or records in the form of a chain. Each member of the chain is also known as a "**node**". Node consists of two fields, one contains the actual record/data and the second field consists of a link to the next member in the chain.
The principal benefit of a linked list over a conventional array is that the order of the linked items may be different from the order that the data items are stored in memory or on disk. For that reason, linked lists allow insertion and removal of nodes at any point in the list, with a constant number of operations

*Each item or member of a linked list is called as **node** or **element**. *Each node contains the address or link to the next node and is known as **next pointer** or **next link**. *The remaining fields are known as the **data**, **value**, **cargo**, **payload** or **information** fields. *Head is the first node and tail is the last node (or vice-verse). *Address to the head node gives access to the entire list

Linked Lists

We can see that a list type container offers a powerful alternative to the array. A data structure that can begin to give the capabilities that we"re looking for is called a linked list. The linked list is structured much like a chain. If we want to make the chain longer, we add a link. If we want it shorter, we remove a link. It"s easy to add or to remove links at either end of the chain; we can add or remove elements of linked list in a similar way. For the chain and the linked list, adding or removing elements in middle of the structure is more difficult.

Designing a Linked List

As we examine the linked list in greater detail, we will also be illustrating some of the thought process that one goes through in executing the design of any software or hardware module. As a way to manage the complexity of a new design, we begin by looking for related concepts that we may already understand. While the linked list is not a particularly complex design, the steps we use remain appropriate for those designs that arein the opening discussion on lists, we've identified some of the desirable high level requirements. Let's now begin to formalize the design.

The first steps involve taking a look at the design from the outside, that is, we take an external view. Such a view reflects how our clients will use our system. We then move to the inside as we begin to develop the system that will give rise to the required features and capabilities. We start by specifying the interface and then proceed to the implementation. We first ask what then follow with how. The use cases for the list that we identified earlier give the first cut at our external interface.

The Linked List combining collections of nodes in an organized way gives a schematic picture of the evolving linked list concept in Figure 3.3. The first element or node is called the head. The last is called the tail. We say that the tail is grounded (NULL) to identify the end of the list. We can see that we can move through the linked list by following the next reference from one node to next. When we reach a node whose next reference is NULL, we know that we have reached the end of the list. When the head has a NULL value, we now that the list is empty, it contains no elements.

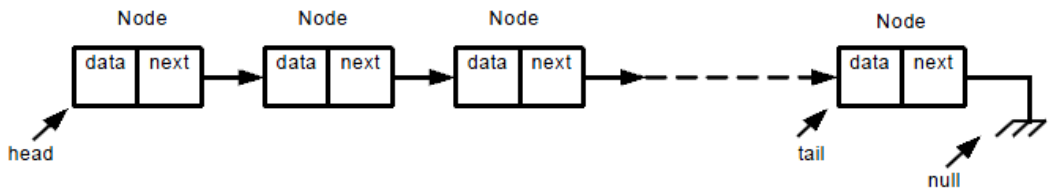
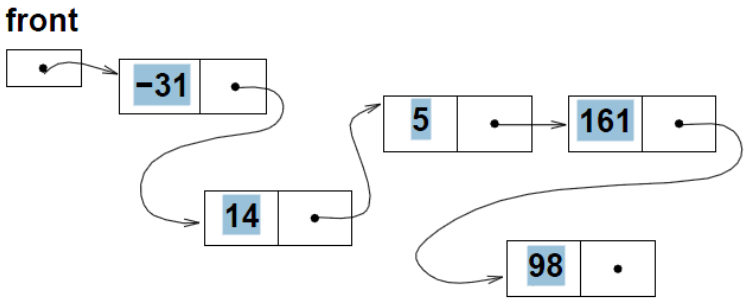


Figure 3.3

High Level Model of a Linked List Data Type

A simpler picture



Operations on Linked List

• Create a list – createList()
• Add an element – addElement()
• Delete an element – deleteElement()
• Modify an element – modifyElement()
• Retrieve an element – getElement()

Let‟s first look at each from a high level perspective. We assume for the moment that error conditions are not being managed

Types of Linked Lists

1 Single or Singly Linked Lists
2 Double or Doubly Linked Lists
3 Circular Singly Linked Lists
4 Circular Doubly Linked Lists

4.2Singly linked list with operations in lists

Creating a Linked List

Creating a linked list in an embedded context can be tricky. Generally we don‟t have the luxury of dynamic memory allocation. The root of the list must be created at compile time; the Node instances must be created then as well. We hold them in a designated part of memory until we need them.

4.3 Creating ordered list, merging two ordered list

When we add or delete an element to/from the linked list, we must consider 3 cases

1. Add/Delete at tail

2. Add/Delete at head

3. Add/Delete in middle

Let‟s see how to do each of these.

1. Adding at the Tail

This one is rather straight forward. We simply follow the next references to the end of the list then change the last next reference to refer to the new node and set the new node next reference to point to NULL (unless it was initialized that way).

MASTER COPY

2. Adding at the Head

If the list is empty, then set the head reference to refer to the new node. Otherwise, set the new node‟s next reference to the value of the head reference and set the head reference to refer to the new node.

3. Adding in the Middle

Start with the head reference and follow the next references to the position where the new node is to be added. Set the new node‟s next reference to the value of the next reference of the current node. Then, set the current node‟s next reference to refer to the new node.

4. Removing from the Tail

Follow the list to the end and change the next reference of the node preceding the last node to NULL.

5. Removing at the Head

If the list is empty, signal an error. If the list contains only a single element, set the head reference to NULL. Otherwise, set the value of the head reference to the value of the next reference in the first element.

6. Removing from the Middle

Follow the list to the position one before where the node is to be removed. Set the current node‟s next reference to the value of the next reference of the following node.

7. Modifying an Element

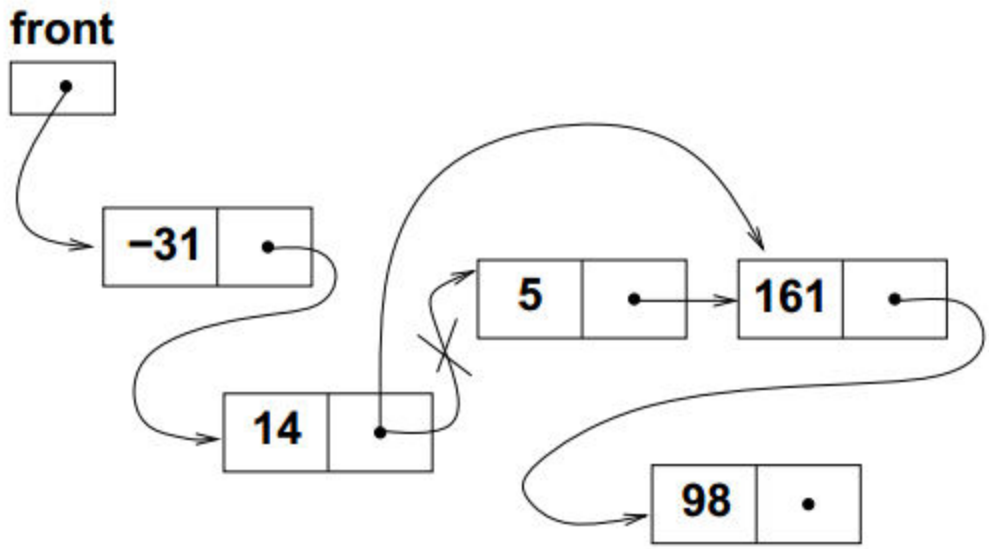
Start with the head reference and follow the next references to the position of the specified node. Change the node‟s value.

8. Retrieve an Element

Start with the head reference and follow the next references to the position of the specified node. Get the node‟s value and return it.

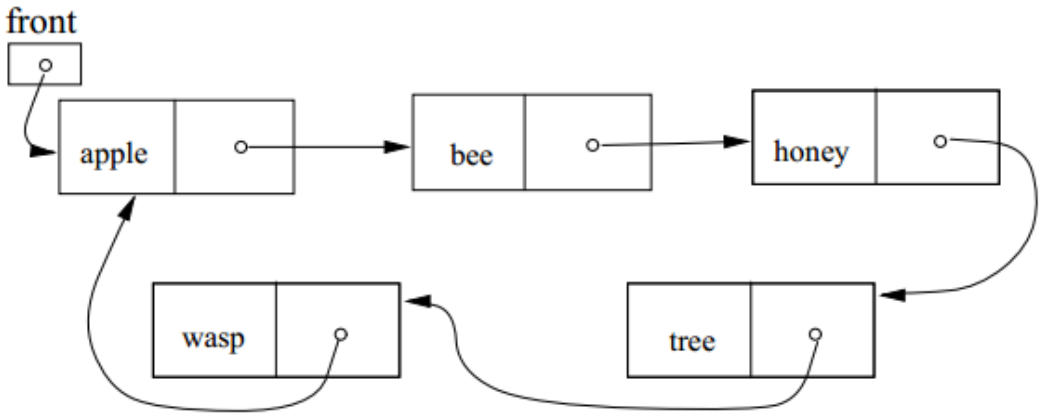
MASTER COPY

Deletion from a Linked List



4.4Circular single linked list

The last element contains a reference to the first element, rather than null in case of Circular Linked List
Circular double linked list can be traversed clockwise or anticlockwise. In this list all nodes are linked as shown in the below figure. The only difference between double linked list and circular double linked list is last node points to first node in the list, first node points to last node instead of pointing to NULLs.
here is the c structure for a node in a circular double linked list and it is same as double linked list. struct Node { void* info; *//data* Node* next; *//used to point to next node* Node* prev; *//used to point to previous node* }; and in this implementation as well (earlier post double linked list also doen"t maintain special nodes), we are not using any special nodes(head or tail), so while operating on the list one must be careful to maintain logical head or tail according to the need.



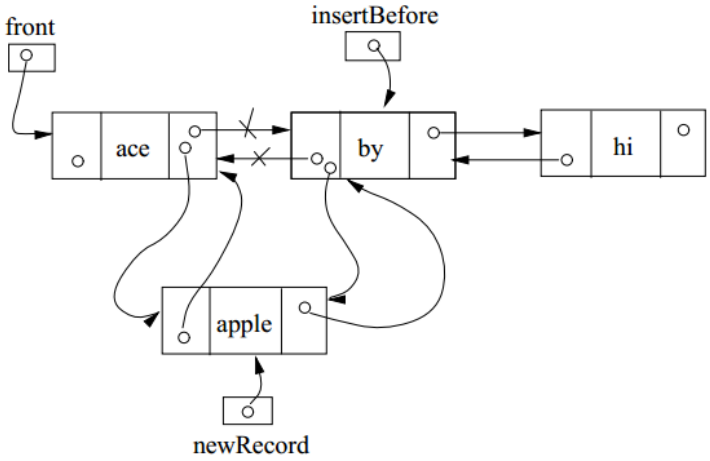
4.5 Circular single linked list with header node

The Circular Doubly Linked Lists consist minimum of two link fields, and one is to point the address of next node and the other one is to point the address of previous node in the list. In this there is no node link will be having NULL value. The first node previous or back link will point to the address of last node and last next will point to the address of the first node in the list.

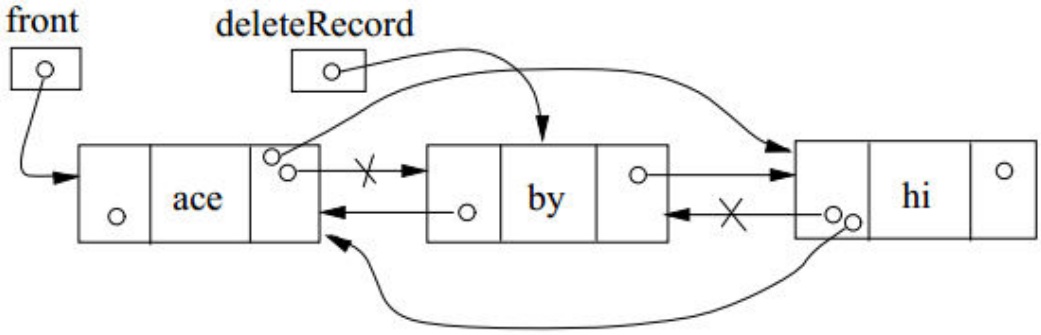
4.6Doubly linked list and Circular double linked list

Each element keeps both a frontwards and a backwards reference.
This could be useful for a list presented on screen, so we can easily write code to scroll up and down. When traversing a doubly-linked list to prepare for insertion or deletion, there is no need to use both a current and previous reference: each node has a built-in reference to the previous element

Insertion with a doubly-linked list:



Deletion with a doubly-linked list



Implementation of Circular Doubly Linked List in C

```
#include<stdio.h>
#include<conio.h>
struct circular
{
int i;
struct circular *front;
struct circular *back;
};
struct circular *temp;
struct circular *head;
struct circular *p;
struct circular *mid;
struct circular *move;
int cnt=0;
void create(void);
void insert(void);
void display(void);
void del(void);
void main()
{
int ch=0;
clrscr();
while(ch!=5)
{
printf("\n1.CREATE");
printf("\n2.INSERT");
printf("\n3.DELETE");
printf("\n4.DISPLAY");
printf("\n5.EXIT");
scanf("%d",&ch);
if(ch==1)
{
create();
cnt++;
cnt++;
}
if(ch==2)
{
insert();
cnt++;
}
if(ch==3)
{
del();
cnt--;
}
if(ch==4)
{
display();
}
```

MASTER COPY

```
if(ch==5)
{
break;
}
}
getch();
}
void create()
{
head=(struct circular *)malloc(sizeof(struct circular));
head->back=head;
head->front=head;
printf("ENETER THE DATA");
scanf("%d",&head->i);
temp=head;
temp->back=(struct circular *)malloc(sizeof(struct circular));
temp=temp->back;
temp->back=head;
head->front=temp;
printf("ENETER THE DATA");
scanf("%d",&temp->i);
}
void insert()
{
int add,t;
printf("\n\t ENTER ANY NUMBER BETWEEN 1 AND %d",cnt);
scanf("%d",&add);
p=head;
t=1;
while(t<add)
{
p=p->back;
t++;
}
clrscr();
mid=(struct circular *)malloc(sizeof(struct circular));
printf("\n\n\nENETER THE DATA");
scanf("%d",&mid->i);
mid->back=p->back;
p->back=mid;
p->back->front=mid;
mid->front=p;
}
void display()
{
p=head;
printf("%\nd-->",p->i);
p=p->back;
while(p!=head)
{
printf("\n%d-->",p->i);
```

MASTER COPY

```
p=p->back;
}
}
void del(void)
{
int add,t;
printf("\n\n\t ENTER ANY NUMBER BETWEEN 1 AND %d",cnt);
scanf("%d",&add);
p=head;
t=1;
while(t<add-1)
{
p=p->back;
t++;
}
clrscr();
mid=p->back;
p->back=mid->back;
mid->back->front=p;
free(mid);
}
```

Linked Lists Vs Arrays

1. Arrays are contiguous

2. In an array, the elements have to be in a contiguous (connected and sequential) portion of memory.

3. Memory immediately next to the array may already be in use for something else.

4. So programming languages don't generally provide for arrays that can grow and shrink in place.

Linked lists are not contiguous

1. In a linked list, the reference in each node says where to and the next one.

2. So the nodes can be all over memory.

3. With an array, we must choose a size once and for all. (This can waste memory, or we could run out of spaces).

4. With a linked list, we can add and remove elements at will. (Each one does take up a little more space for its link field, and it may take time to `get to' the insertion / removal spot).

5. With an array, we can immediately access the nth element.

6. With a linked list, we cannot. We have to follow n pointers

MASTER COPY

4.7 Applications of Linked List

There are several applications of linked lists. Out of these enormous applications we choose to present two applications, which seem to be sufficient to explain the linked-list functions. First we will show a pointer implementation of sparse polynomials which may be considered as an application of singly linked list with fixed head node. Next we present an application of circular doubly linked list with fixed head: large integer arithmetic.
The main Applications of Linked Lists are

1. For representing Polynomials It means in addition/subtraction /multiplication.. of two polynomials. Eg:p1=2x^2+3x+7 and p2=3x^3+5x+2 p1+p2=3x^3+2x^2+8x+9

2. In Dynamic Memory Management In allocation and releasing memory at runtime.

3. In Symbol Tables In Balancing parenthesis

4. Representing Sparse Matrix

Linked List, Advantages and Disadvantages of linked list

Linked List:-
A linked list is the chain of nodes or date items connected by pointers and each node contains a pointer pointing to the address of next node or data item. A linked list is an ordered list of data items.
A link list is a collection of records of the same data type, and all such records are connected through pointers each record in a linked list has a pointer field pointing to the next record in the list. The pointer field in the last record of the list is assigned the null value. A pointer variable is used to point to the first record in the list.

4.7.1 Addition of two long integers

```
#include<stdio.h>
#include<stdlib.h>

/* Linked list node */
struct Node
{
   int data;
   struct Node* next;
};

/* Function to create a new node with given data */
struct Node *newNode(int data)
{
   struct Node *new_node = (struct Node *) malloc(sizeof(struct Node));
   new_node->data = data;
   new_node->next = NULL;
   return new_node;
```

MASTER COPY

```
}

/* Function to insert a node at the beginning of the Doubly Linked List */
void push(struct Node** head_ref, int new_data)
{
   /* allocate node */
   struct Node* new_node = newNode(new_data);

   /* link the old list off the new node */
   new_node->next = (*head_ref);

   /* move the head to point to the new node */
   (*head_ref)    = new_node;
}

/* Adds contents of two linked lists and return the head node of resultant list */
struct Node* addTwoLists (struct Node* first, struct Node* second)
{
   struct Node* res = NULL; // res is head node of the resultant list
   struct Node *temp, *prev = NULL;
   int carry = 0, sum;

   while (first != NULL || second != NULL) //while both lists exist
   {
      // Calculate value of next digit in resultant list.
      // The next digit is sum of following things
      // (i)  Carry
      // (ii) Next digit of first list (if there is a next digit)
      // (ii) Next digit of second list (if there is a next digit)
      sum = carry + (first? first->data: 0) + (second? second->data: 0);

      // update carry for next calulation
      carry = (sum >= 10)? 1 : 0;

      // update sum if it is greater than 10
      sum = sum % 10;

      // Create a new node with sum as data
      temp = newNode(sum);

      // if this is the first node then set it as head of the resultant list
      if(res == NULL)
         res = temp;
      else // If this is not the first node then connect it to the rest.
         prev->next = temp;

      // Set prev for next insertion
      prev  = temp;

      // Move first and second pointers to next nodes
      if (first) first = first->next;
```

MASTER COPY

```
        if (second) second = second->next;
    }

    if (carry > 0)
      temp->next = newNode(carry);

    // return head of the resultant list
    return res;
}

// A utility function to print a linked list
void printList(struct Node *node)
{
    while(node != NULL)
    {
        printf("%d ", node->data);
        node = node->next;
    }
    printf("\n");
}

/* Driver program to test above function */
int main(void)
{
    struct Node* res = NULL;
    struct Node* first = NULL;
    struct Node* second = NULL;

    // create first list 7->5->9->4->6
    push(&first, 6);
    push(&first, 4);
    push(&first, 9);
    push(&first, 5);
    push(&first, 7);
    printf("First List is ");
    printList(first);

    // create second list 8->4
    push(&second, 4);
    push(&second, 8);
    printf("Second List is ");
    printList(second);

    // Add the two lists and see result
    res = addTwoLists(first, second);
    printf("Resultant list is ");
    printList(res);

   return 0;
}
```

MASTER COPY

4.7.2 Evaluation of polynomial

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h> // For power function
/*<————————GLOBAL DECLARATION————————>*/
typedef struct node
{
int coef;
int power; //Power to X
struct node *link;
}*NODE;
NODE First = NULL;
//Creating the polynomial
void Create()
{
NODE Temp, New, Prev; //Don't take as "new" because "new" is a kayword
int coef, power;
printf("Enter the coef and power terminated by coef 0 \n");
scanf("%d %d", &coef, &power);
while(coef != 0)
{
New = (NODE)malloc(sizeof(struct node));
New->coef = coef;
New->power = power;
New->link = NULL; //As it is the First node its link must be NULL
if(First == NULL) //Like First == NULL in normal list
First = New ;
/* If First is NULL, then there is no list, so New is the First Node inserted */
else //If the list has some nodes
{
if(power > First->power) /*To place the x to the power*/
{
New->link = First; //If it is the highest power
First = New;
}
else
{
Temp = First;
while( (Temp!= NULL) && (power < Temp->power) ) //if power to x is less
{
Prev = Temp;
Temp = Temp->link; //To traverse
}
if(Temp == NULL)    //Its true when last node reached, means least power
{
Prev->link = New;
New->link = Temp; //New->link = NULL;
}
```

MASTER COPY

```
else //For Intermediate Power
{
Prev->link = New;
New->link = Temp;
}
}
}
scanf("%d %d", &coef, &power);
}
}
//Evaluation of the Polynomial
int Evaluate()
{
NODE Temp;
int result = 0, x;
printf("Enter the value of X in the Polynomial  \n");
scanf("%d", &x);
Temp = First;
while(Temp!= NULL) //Traversing
{
result = result + Temp->coef * pow(x, Temp->power);
Temp = Temp->link;
}
return result;
}
//Display Function
void Display()
{
NODE Temp;
if(First == NULL)
{
printf("Empty Polynomial \n");
return;
}
Temp = First;
while(Temp!= NULL) //Traversing
{
printf("|%d|%d| –>", Temp->coef, Temp->power);
Temp = Temp->link;
}
printf("NULL"); // Just to say polynomail ended
}
/*<————————MAIN FUNCTION————————>*/
int main()
{
int choice, result;
for(;;)
{
printf("\n1.Create Polynomial\n2.Evaluate\n3.Display\n4.Exit\n");
printf("Enter your choice : \n");
scanf("%d", &choice);
```

MASTER COPY

```
switch(choice)
{
case 1:
Create();
printf("Polynomial After Creation \n");
Display();
break;
case 2:
result = Evaluate();
printf("Result is %d", result);
break;
case 3:
printf("Polynomial is \n");
Display();
break;
case 4:
exit(0);
default:
printf("Invalid option\n\n");
break;
}
}
}
```

4.7.3 Addition of two polynomials

```
#include<stdio.h>
#include<stdlib.h>

/**
 * The structure for the polynomial
 * This is a linked list with two variable
 * int coeff The Coefficient
 * int pow The power of x
 */
typedef struct link {
    int coeff;
    int pow;
    struct link * next;
} my_poly;

/** The prototypes */
void my_create_poly(my_poly **);
void my_show_poly(my_poly *);
void my_add_poly(my_poly **, my_poly *, my_poly *);

/**
 * The simple menu driven main function
 */
int main(void) {
```

MASTER COPY

```
    int ch;
    do {
        my_poly * poly1, * poly2, * poly3;

        printf("\nCreate 1st expression\n");
        my_create_poly(&poly1);
        printf("\nStored the 1st expression");
        my_show_poly(poly1);

        printf("\nCreate 2nd expression\n");
        my_create_poly(&poly2);
        printf("\nStored the 2nd expression");
        my_show_poly(poly2);

        my_add_poly(&poly3, poly1, poly2);
        my_show_poly(poly3);

        printf("\nAdd two more expressions? (Y = 1/N = 0): ");
        scanf("%d", &ch);
    } while (ch);
    return 0;
}

/**
 * The create polynomial function
 * @param my_poly ** node The pointer to the head of the polynomial
 * We will modify the parameter and will store the base address
 * @return void
 */
void my_create_poly(my_poly ** node) {
    int flag; //A flag to control the menu
    int coeff, pow;
    my_poly * tmp_node; //To hold the temporary last address
    tmp_node = (my_poly *) malloc(sizeof(my_poly)); //create the first node
    *node = tmp_node; //Store the head address to the reference variable
    do {
        //Get the user data
        printf("\nEnter Coeff:");
        scanf("%d", &coeff);
        tmp_node->coeff = coeff;
        printf("\nEnter Pow:");
        scanf("%d", &pow);
        tmp_node->pow = pow;
        //Done storing user data

        //Now increase the Linked on user condition
        tmp_node->next = NULL;

        //Ask user for continuation
        printf("\nContinue adding more terms to the polynomial list?(Y = 1/N = 0): ");
        scanf("%d", &flag);
```

MASTER COPY

```
        //printf("\nFLAG: %c\n", flag);
        //Grow the linked list on condition
        if(flag) {
            tmp_node->next = (my_poly *) malloc(sizeof(my_poly)); //Grow the list
            tmp_node = tmp_node->next;
            tmp_node->next = NULL;
        }
    } while (flag);
}

/**
 * The show polynomial function
 * Prints the Polynomial in user readable format
 * @param my_poly * node The polynomial linked list
 * @return void
 */
void my_show_poly(my_poly * node) {
    printf("\nThe polynomial expression is:\n");
    while(node != NULL) {
        printf("%dx^%d", node->coeff, node->pow);
        node = node->next;
        if(node != NULL)
            printf(" + ");
    }
}

/**
 * The polynomial add function
 * Adds two polynomial to a given variable
 * @param my_poly ** result Stores the result
 * @param my_poly * poly1 The first polynomial expression
 * @param my_poly * poly2 The second polynomial expression
 * @return void
 */
void my_add_poly(my_poly ** result, my_poly * poly1, my_poly * poly2) {
    my_poly * tmp_node; //Temporary storage for the linked list
    tmp_node = (my_poly *) malloc(sizeof(my_poly));
    tmp_node->next = NULL;
    *result = tmp_node; //Copy the head address to the result linked list

    //Loop while both of the linked lists have value
    while(poly1 && poly2) {
        if (poly1->pow > poly2->pow) {
            tmp_node->pow = poly1->pow;
            tmp_node->coeff = poly1->coeff;
            poly1 = poly1->next;
        }
        else if (poly1->pow < poly2->pow) {
            tmp_node->pow = poly2->pow;
            tmp_node->coeff = poly2->coeff;
            poly2 = poly2->next;
```

MASTER COPY

```
        }
        else {
            tmp_node->pow = poly1->pow;
            tmp_node->coeff = poly1->coeff + poly2->coeff;
            poly1 = poly1->next;
            poly2 = poly2->next;
        }

        //Grow the linked list on condition
        if(poly1 && poly2) {
            tmp_node->next = (my_poly *) malloc(sizeof(my_poly));
            tmp_node = tmp_node->next;
            tmp_node->next = NULL;
        }
    }

    //Loop while either of the linked lists has value
    while(poly1 || poly2) {
        //We have to create the list at beginning
        //As the last while loop will not create any unnecessary node
        tmp_node->next = (my_poly *) malloc(sizeof(my_poly));
        tmp_node = tmp_node->next;
        tmp_node->next = NULL;

        if(poly1) {
            tmp_node->pow = poly1->pow;
            tmp_node->coeff = poly1->coeff;
            poly1 = poly1->next;
        }
        if(poly2) {
            tmp_node->pow = poly2->pow;
            tmp_node->coeff = poly2->coeff;
            poly2 = poly2->next;
        }
    }

    printf("\nAddition Complete");
}
```

4.8 Advantage and disadvantage of different lists

Advantages of linked list:-

 It is not necessary to know in advance the number of elements to be stored in the list and therefore, need not allocate. d as and when necessary.

 In a linked list, insertions and deletions can be handled efficiently without fixing the size of the memory in advance.

MASTER COPY

 An important advantage of linked lists over arrays is that the linked list uses exactly as much memory as it needs, and can be made to expand to fill all available memory locations if needed.

Disadvantages:-

 The traversal is sequential.
 Increased overhead for storing pointers for linking the data items.

Chapter-5 TREES

5.1 Introduction, Definition, storage representation of a binary tree, Terminologies associated with trees

Definition

Tree is a non-linear data structure, which is a collection of zero or more nodes and a finite set of directed lines called branches that connects the nodes. The first node in a tree is called as root node, and remaining nodes are partitioned into various sub trees. The nodes attached to the left of root is called as left sub tree and the nodes are connected to the right of root is called as right sub tree.

The nodes are connected to the left of root is called as left sub tree and the nodes are connected to the right of the tree called as right sub tree.

The trees are basically classified into two categories, **Binary trees** and **General trees**.

Binary Tree

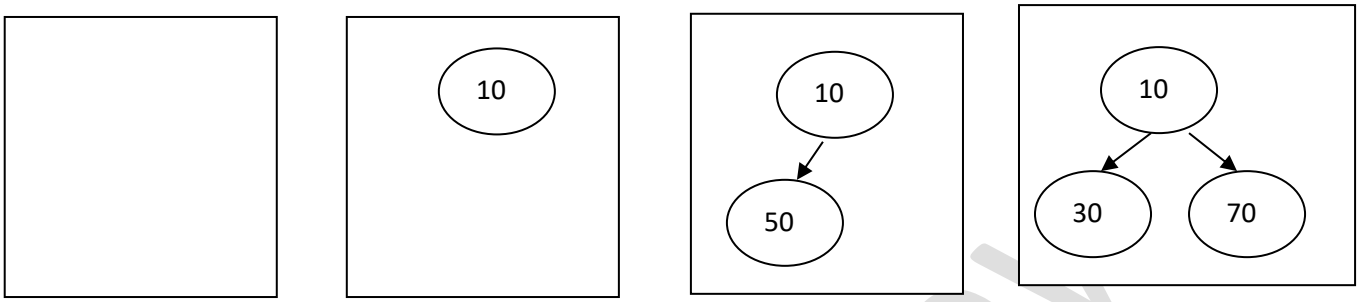
Binary tree is a tree which is a collection of zero or more nodes and a finite set of directed lines called as branches that connect the nodes. Where every node in a tree may have maximum of two children. A node in a tree can have either zero, one or two children. If every node in a tree satisfies this condition then the tree is called as

MASTER COPY

binary tree.

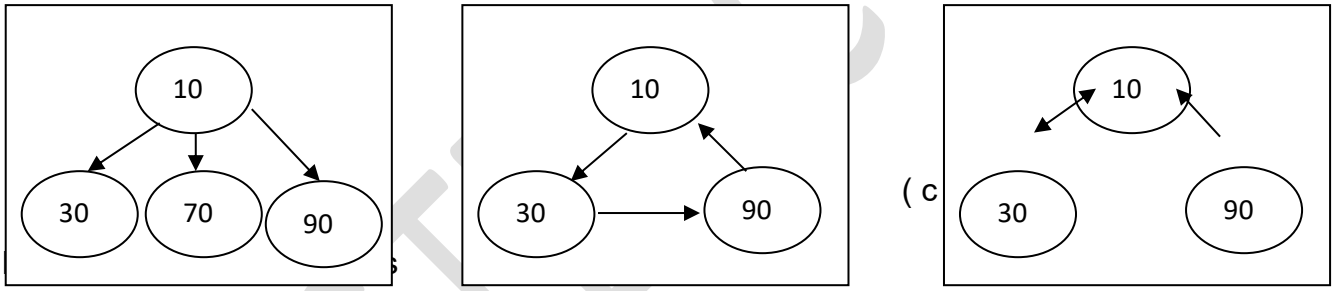
An empty tree with no node exists is also a binary tree

Diagram - Binary Tress



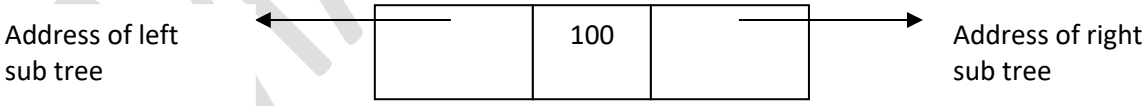
General Tree

A tree which doesn't poses any definite structure then the tree is called as General tree or Non Binary Tree



The Structure of Node in a Tree

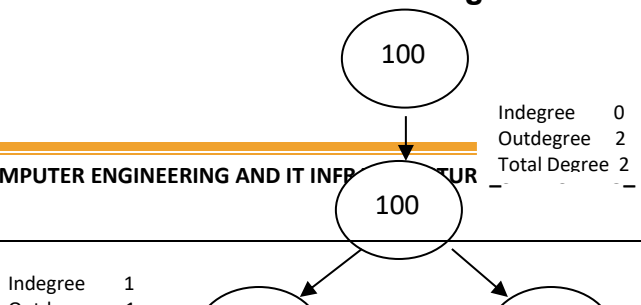
A binary tree has either zero, one or two sub trees exist. If any node has more than two sub trees then it is not a binary tree. The pictorial representation of a typical node in a tree is



Terminologies Associated with Trees

1. Degree

The number of branches associated with each node in a tree is called as degree of a node. When a branch is directed towards the node is called as **Indegree** and the branches away from a node are called as **Outdegree.**



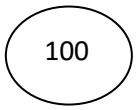


Figure 3.1

2. Root Node

The node with indegree zero is called as root node. Generally it is the first node that created in the tree. In figure 3.2 the node with value 100 is the root node.

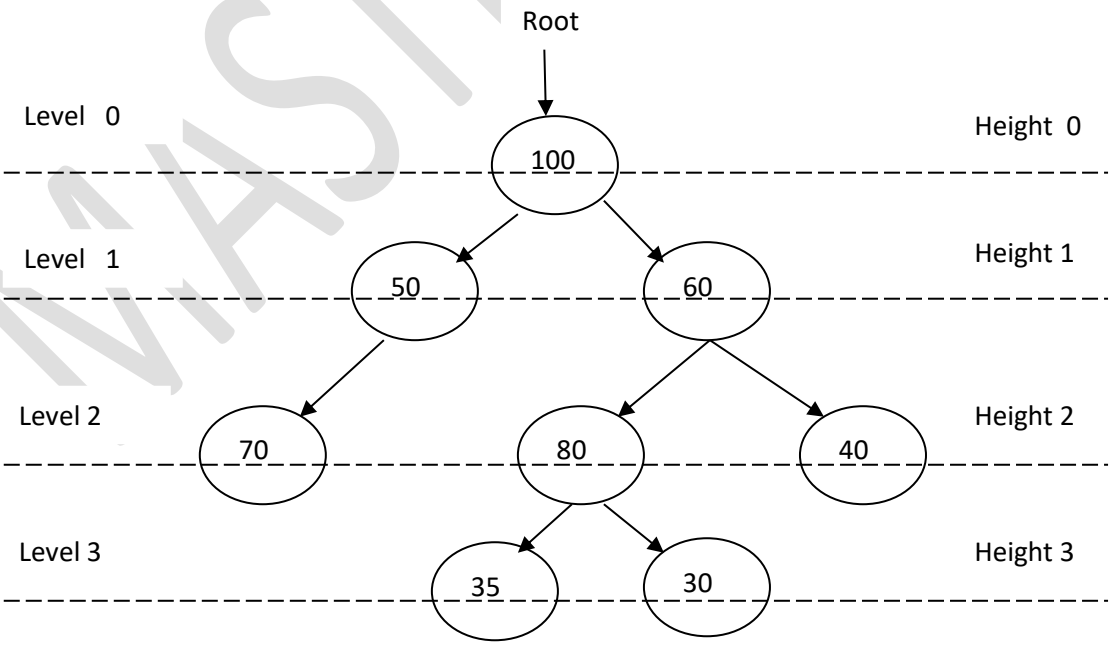


Figure 3.2

3. Child Node

MASTER COPY

The node which are all reachable from node x using only one edge are called as children of node x and node x is the parent of all those children. In the above example 50 and 60 are children's of node 100. And node 80 and 40 are the children of node 60.

4. Siblings

The nodes with same parent are called as siblings. For example.

- 50 and 60 are siblings since they have same parent 100
- 80 and 40 are siblings since they have same parent 60
- 30 and 30 are siblings since they have same parent 80

5. Ancestors

The nodes in the path from root to the specified node x, are all ancestors of node x.

Eg. 100 is the ancestor of 50
60 is the ancestor of node 100
50 and 100 are the ancestor of node 70
60 and 100 are the ancestor of node 80 and 40
80, 60 and 100 are the ancestor of node 30 and 35

6. Descendents

The nodes in the path below the parent are called as descendents. In other words the nodes that are all reachable from a node x are all the descendents of node x.

Ex:
All the nodes below 100 are descendents of node 100
All the nodes below 60 are descendents of node 60 (80, 40, 35, 30)

6. Left descendents

The nodes that lie towards the left sub tree of node x are called left descendents.

- 50 and 70 are left descendents of 100
- 80, 35 and 70 are left descendents of 60
- 35 and 80 are left descendents of 60

7. Right descendents

The nodes that lie towards the right sub tree of node x are called Right descendents.

- 60,80, 40, 35, and 30 are right descendents of 100
- 30 is the right descendents of 80
- 40 is the right descendents of 60

8. Parent Node

The nodes having left sub tree and right sub tree or both is said to be the parent node for left sub tree and right sub tree.

Node 100 is the parent of 50 and 60
50 is the parent node for 70
60 is the parent node for 80 and 40

MASTER COPY

9. Leaf Node

The node in a tree with outdegree zero is called as leaf node. Or the node with empty left child and empty right child. Or node with no children is also called as leaf node. Leaf node is also called as terminal nodes or external nodes.

Ex : 70, 35, 30, 40 are leaf node in the Tree Fig 3.2

10. Internal Nodes

The nodes except the leaf are called as external nodes. Example 100, 50, 60 and 80 are the internal nodes in Tree fig 3.2

11. External Nodes

Leaf node is also called as terminal nodes or external nodes.

Ex : 70, 35, 30, 40 are leaf node in the Tree Fig 3.2

12. Level

The distance of a node from the root is called level of the node. In a tree the root has a **level 0,** and level of any other node is one more than the level of its parent.

13. Depth or Height

The height of the tree is defined as the maximum level of any leaf in the tree whether it is in left sub tree or right sub tree.

5.2 Operations on binary tree

```
/* Program Tree Operation insert traversal preorder,post order,inorder*/

#include<stdio.h>
#include<alloc.h>
#include<conio.h>
#include<stdio.h>
struct tree
{
 int info;
 struct tree *left;
 struct tree *right;
};

struct tree *insert(struct tree *,int);
void inorder(struct tree *);
void postorder(struct tree *);
void preorder(struct tree *);
struct tree *delet(struct tree *,int);
struct tree *search(struct tree *);

int main(void)
{

 struct tree *root;
```

MASTER COPY

```
 int choice, item,item_no;
 root = NULL;
 clrscr();
/* rear  = NULL;*/
 do
 {
  do
  {
   printf("\n \t 1. Insert in Binary Tree  ");
   printf("\n\t 2. Delete from Binary Tree ");
   printf("\n\t 3. Inorder traversal of Binary tree");
   printf("\n\t 4. Postorder traversal of Binary tree");
   printf("\n\t 5. Preorder traversal of Binary tree");
   printf("\n\t 6. Search and replace ");
   printf("\n\t 7. Exit ");
   printf("\n\t Enter choice : ");
   scanf(" %d",&choice);
   if(choice<1 || choice>7)
      printf("\n Invalid choice - try again");
  }while (choice<1 || choice>7);
  switch(choice)
  {
   case 1:
  printf("\n Enter new element: ");
  scanf("%d", &item);
  root= insert(root,item);
  printf("\n root is %d",root->info);
  printf("\n Inorder traversal of binary tree is : ");
  inorder(root);
  break;
   case 2:
 printf("\n Enter the element to be deleted : ");
 scanf(" %d",&item_no);
 root=delet(root,item_no);
 inorder(root);
  break;
   case 3:
 printf("\n Inorder traversal of binary tree is : ");
 inorder(root);
 break;
   case 4:
 printf("\n Postorder traversal of binary tree is : ");
 postorder(root);
 break;
   case 5:
 printf("\n Preorder traversal of binary tree is : ");
 preorder(root);
 break;
   case 6:
 printf("\n Search and replace operation in binary tree ");
 root=search(root);
```

MASTER COPY

```
 break;
   default:
  printf("\n End of program ");
   } /* end of switch */
  }while(choice !=7);
 return(0);
}

struct tree *insert(struct tree *root, int x)
{
 if(!root)
  {
    root=(struct tree*)malloc(sizeof(struct tree));
    root->info = x;
    root->left = NULL;
    root->right = NULL;
    return(root);
  }
  if(root->info > x)
     root->left = insert(root->left,x);
  else
   {
     if(root->info < x)
root->right = insert(root->right,x);
   }
   return(root);
 }


 void inorder(struct tree *root)
 {
    if(root != NULL)
   {
     inorder(root->left);
     printf(" %d",root->info);
     inorder(root->right);
   }
   return;
 }

 void postorder(struct tree *root)
 {
    if(root != NULL)
   {
     postorder(root->left);
     postorder(root->right);
     printf(" %d",root->info);
   }
   return;
 }
```

MASTER COPY

```
void preorder(struct tree *root)
{
   if(root != NULL)
  {
    printf(" %d",root->info);
    preorder(root->left);
    preorder(root->right);
  }
  return;
}

/* FUNCTION TO DELETE A NODE FROM A BINARY TREE */
struct tree *delet(struct tree *ptr,int x)
{
 struct tree *p1,*p2;
 if(!ptr)
 {
   printf("\n Node not found ");
   return(ptr);
 }
 else
 {
    if(ptr->info < x)
    {
     ptr->right = delet(ptr->right,x);
     /*return(ptr);*/
    }
    else if (ptr->info >x)
     {
      ptr->left=delet(ptr->left,x);
      return ptr;
     }
     else                /* no. 2 else */
      {
if(ptr->info == x)    /* no. 2 if */
{
 if(ptr->left == ptr->right) /*i.e., a leaf node*/
  {
   free(ptr);
   return(NULL);
  }
 else if(ptr->left==NULL)  /* a right subtree */
{
  p1=ptr->right;
 free(ptr);
 return p1;
}
else if(ptr->right==NULL)   /* a left subtree */
{
p1=ptr->left;
free(ptr);
```

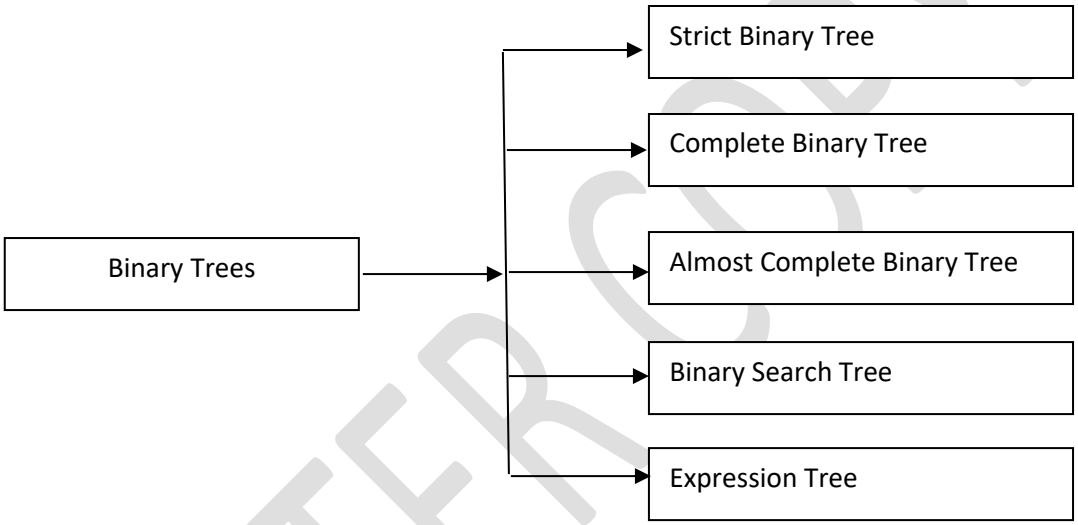
MASTER COPY

```
return p1;
}
else
{
 p1=ptr->right;
 p2=ptr->right;
while(p1->left != NULL)
   p1=p1->left;
p1->left=ptr->left;
free(ptr);
return p2;
}
      }/*end of no. 2 if */
     }/* end of no. 2 else */
  /* check which path to search for a given no. */
 }
  return(ptr);
}
/* function to search and replace an element in the binary tree */
struct tree *search(struct tree *root)
{
 int no,i,ino;
 struct tree *ptr;
 ptr=root;
 printf("\n Enter the element to be searched :");
 scanf(" %d",&no);
 fflush(stdin);
 while(ptr)
 {
  if(no>ptr->info)
     ptr=ptr->right;
  else if(no<ptr->info)
     ptr=ptr->left;
  else
     break;
 }
 if(ptr)
 {
  printf("\n Element %d which was searched is found and is = %d",no,ptr->info);
  printf("\n Do you want replace it, press 1 for yes : ");
  scanf(" %d",&i);
  if(i==1)
  {
   printf("\n Enter new element :");
   scanf(" %d",&ino);
   ptr->info=ino;
  }
  else
    printf("\n\t It's okay");
 }
 else
```

MASTER COPY

```
  printf("\n Element %d does not exist in the binary tree",no);
 return(root);
}
```

Types of Binary Trees



Binary Trees

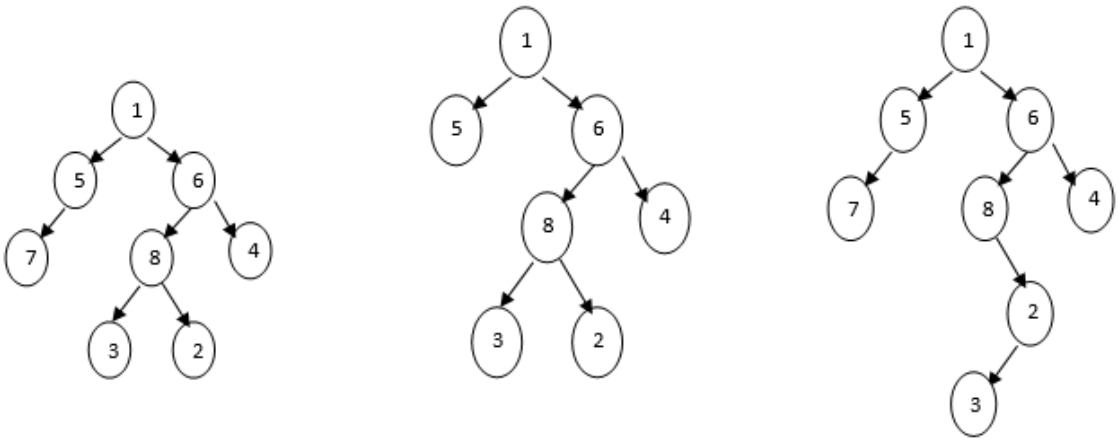


Fig (a) Fig (b) Fig (c)

1. Strict Binary Tree

If outdegree of every node in a tree has either zero or two nodes, then the tree is said to be strict binary tree. Ie each node in the tree can have maximum two children or no children.

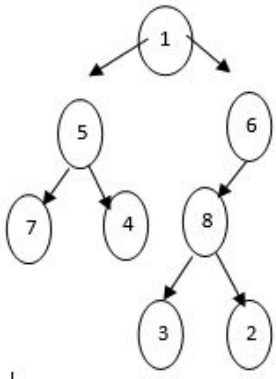


Fig (a)

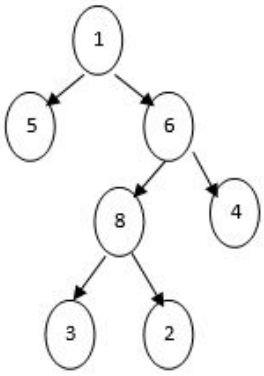


Fig (b)

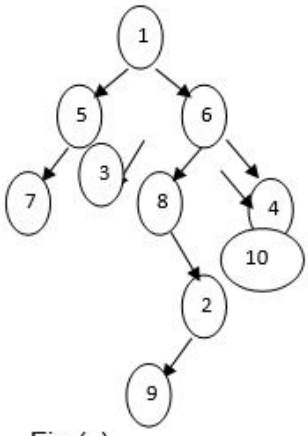


Fig (c)

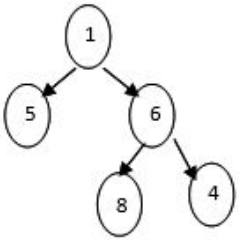


Fig (d)

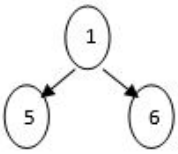


Fig (e)

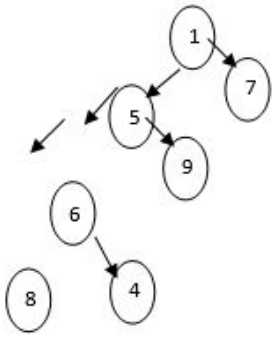


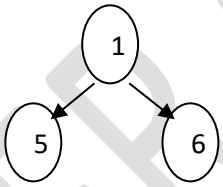
Fig (f)

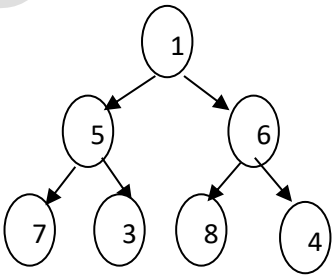
In the above example trees fig (a-f) all nodes in the tree has either zero or two children. If all nodes satisfies this condition then this tree is called as **strict binary tree.**

2. Complete Binary Tree

A strict binary tree in which the number of nodes at any level **i** is 2^i, and then the tree is said to a Complete Binary Tree. Ie a binary tree in which the '**n**' th Level must have 2^n nodes then the tree will be called as Complete Binary Tree.

A Complete Binary tree will be binary tree and strict binary tree too.

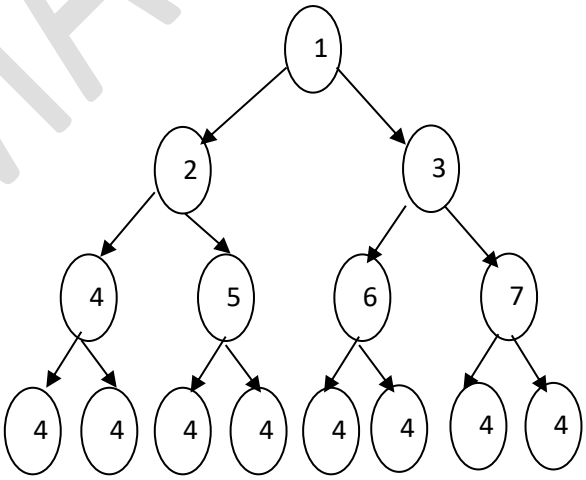




Level 0 = 2^0 Nodes Level 1 = 2^1 Nodes Level 2 = 2^2 Nodes

Level 0 = 1 node Level 1 = 2 nodes Level 2 = 4 nodes

Fig (a) Fig (b) Fig (c)



Level 2 = 2^3 Nodes
Level 3 = 8 nodes

Fig (d)

MASTER COPY

A tree with n level exists, and n^{th} level we have 2^n Nodes then these trees are called as ***Complete binary tree.***

3. Almost Complete Binary Tree

A tree with depth 'd' is said to be almost complete binary tree, then it should satisfy following properties

a) The tree must be complete up to level 'd-1', then the total number of nodes at level 'd-1' should be 2^{d-1}.

b) The total number of nodes at level d may be 2^d , if the total number of nodes at level 'd' is less than 2^d , then the number of nodes at level 'd-1' should be 2^{d-1} and in level d the nodes should be present only from left to right.

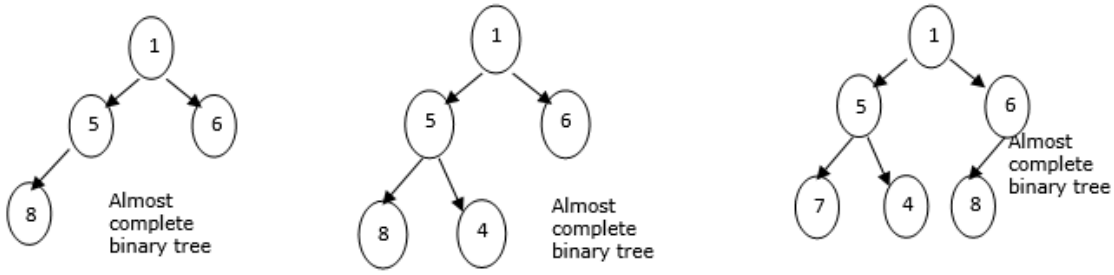


Fig (a) Fig (b) Fig (c)

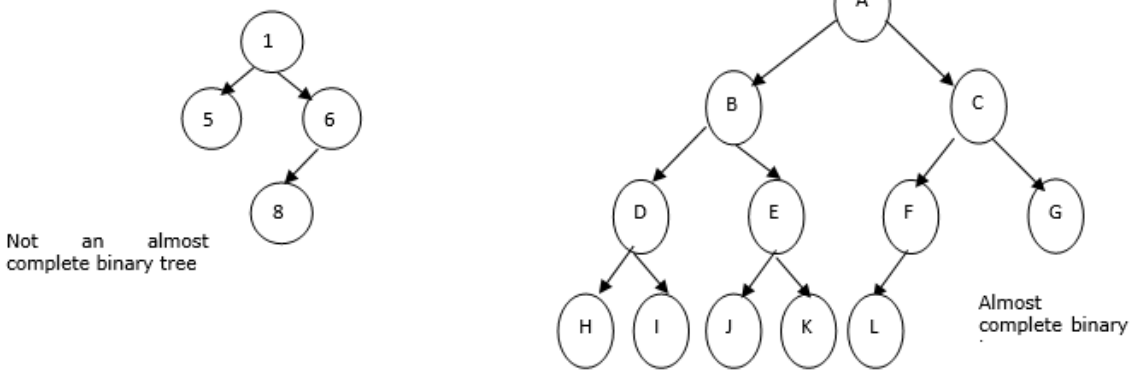


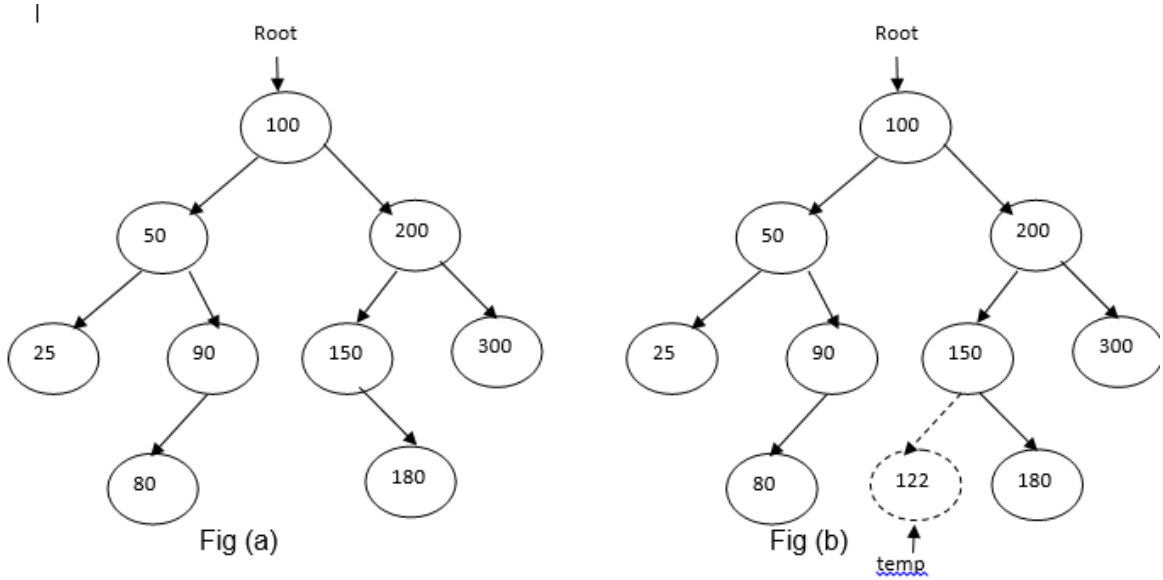
Fig (a) Fig (c)

5.3. Binary Search Tree. (BST)

A **binary search tree** is a binary tree in which every node say x in the tree, elements in the left sub tree are less than info(x). and elements in the right sub tree are greater than equal to info(x). Every node in tree should satisfy this condition if left sub tree or right sub tree exists.

If a binary tree is said to be a Binary Search Tree, then every node in a tree should satisfy following condition.

1. The left sub tree node value must be less than its parent's node.
2. The right sub tree node value must be greater than or equal to its parents node.
3. If every node in a tree must satisfy this condition then the tree is said to be binary search tree.



In fig (b) , a new node (122) is been added into binary search tree to the left of node (150). Since newly inserted node (122) value is less than its root, will be attached on the left sub tree.

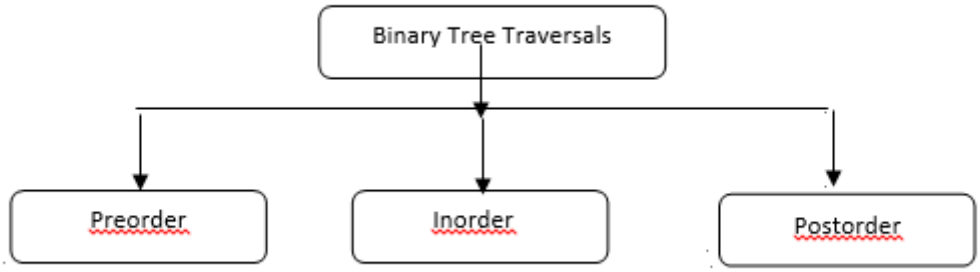
Operations on Trees.

1. Insertion - (Insert a given item into tree)
2. Traversal - (Visiting the nodes in a tree exactly once)
3. Search - (Search for a key item)
4. Copy - (To obtain the exact copy of the given tree)
5. Deletion - (Delete a node from tree)

1. Insertion in Tree

2. Tree Traversals

Traversal is a method of visiting each node in a tree exactly once in a systematic order based on the order. During the traversal we can print or display the info field of every node

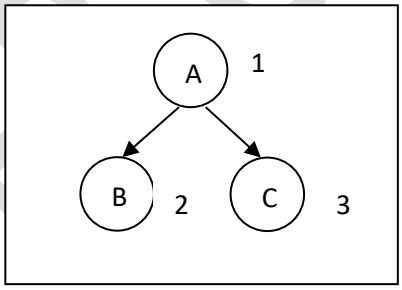


1 Preorder

Preorder traversal in a binary tree can be performed recursively by following order.

1. Visit the root node (Root)
2. Traverse the left sub tree in preorder recursively (Left)
3 Traverse the right sub tree in preorder recursively (right)

```
void preorder( Node root)
{
        if(root ==NULL)
                return ;
        printf("\n%d ", root->data);
        preorder(root->llink);
        preorder(root->rlink);
}
```



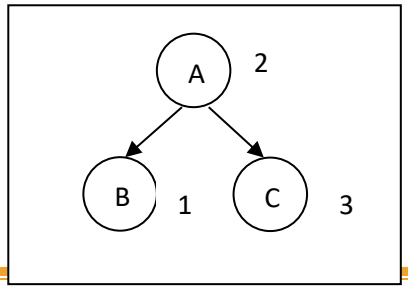
Preorder Traversal = ABC

2 Inorder

Inorder traversal in a binary tree can be performed recursively by following order.

1. Traverse the left sub tree in preorder recursively (Left)
2. Visit the root node (Root)
3. Traverse the right sub tree in preorder recursively (right)

```
void inorder( Node root)
{
        if(root ==NULL)
                return ;
        inorder(root->llink);
        printf("\n%d ", root->data);
        inorder(root->rlink);
```



MASTER COPY

}

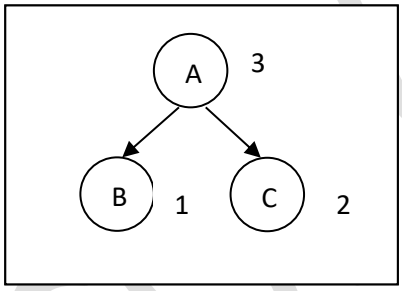
Inorder Traversal = BAC

3 Postorder

Preorder traversal in a binary tree can be performed recursively by following order.

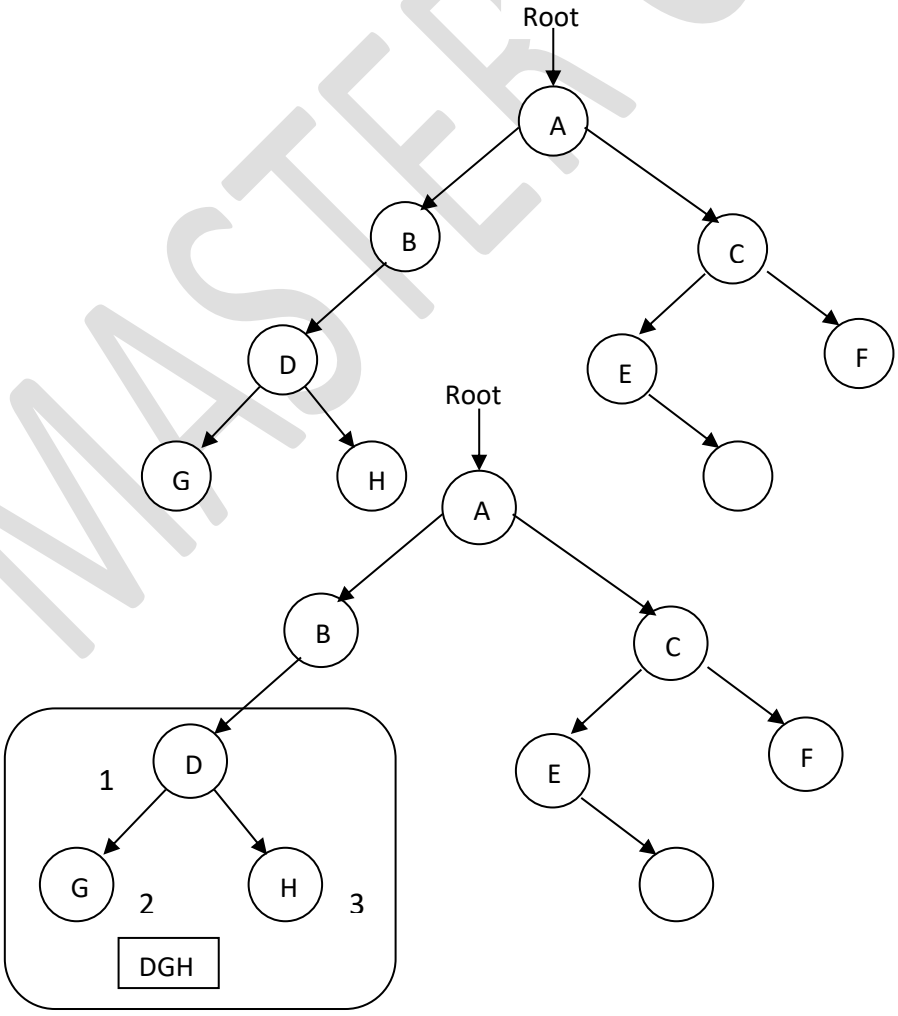
1. Traverse the left sub tree in postorder recursively (Left)
3. Traverse the right sub tree in postorder recursively (right)
2. Visit the root node (Root)

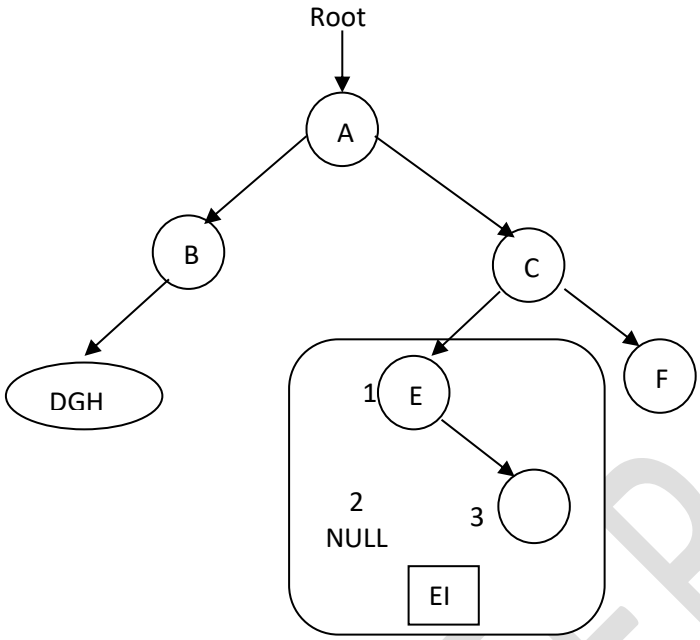
```
void postorder( Node root)
{
        if(root ==NULL)
                return ;
        postorder(root->llink);
        postorder(root->rlink);
        printf("\n%d ", root->data);
}
```

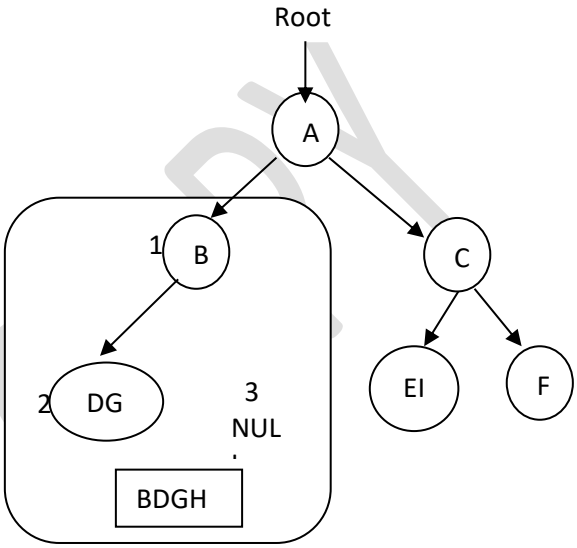


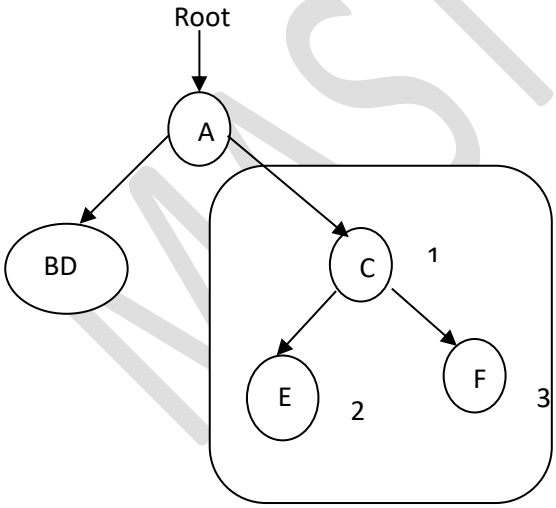
Postorder Traversal = BCA

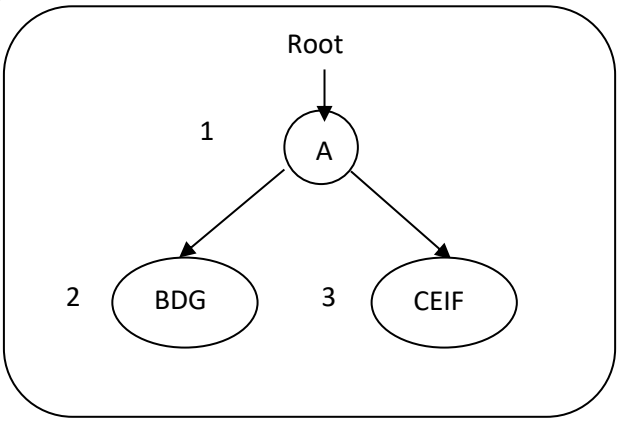
1. Example: Traversal of Preorder (Root, Left, Right)

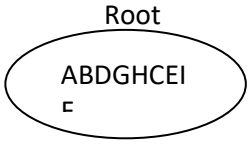



Root
A
B
C
DGH
1
E
F
2
NULL
3
EI


Root
A
1
B
C
2
DG
3
NUL
BDGH
EI
F


Root
A
BD
C
1
E
2
F
3

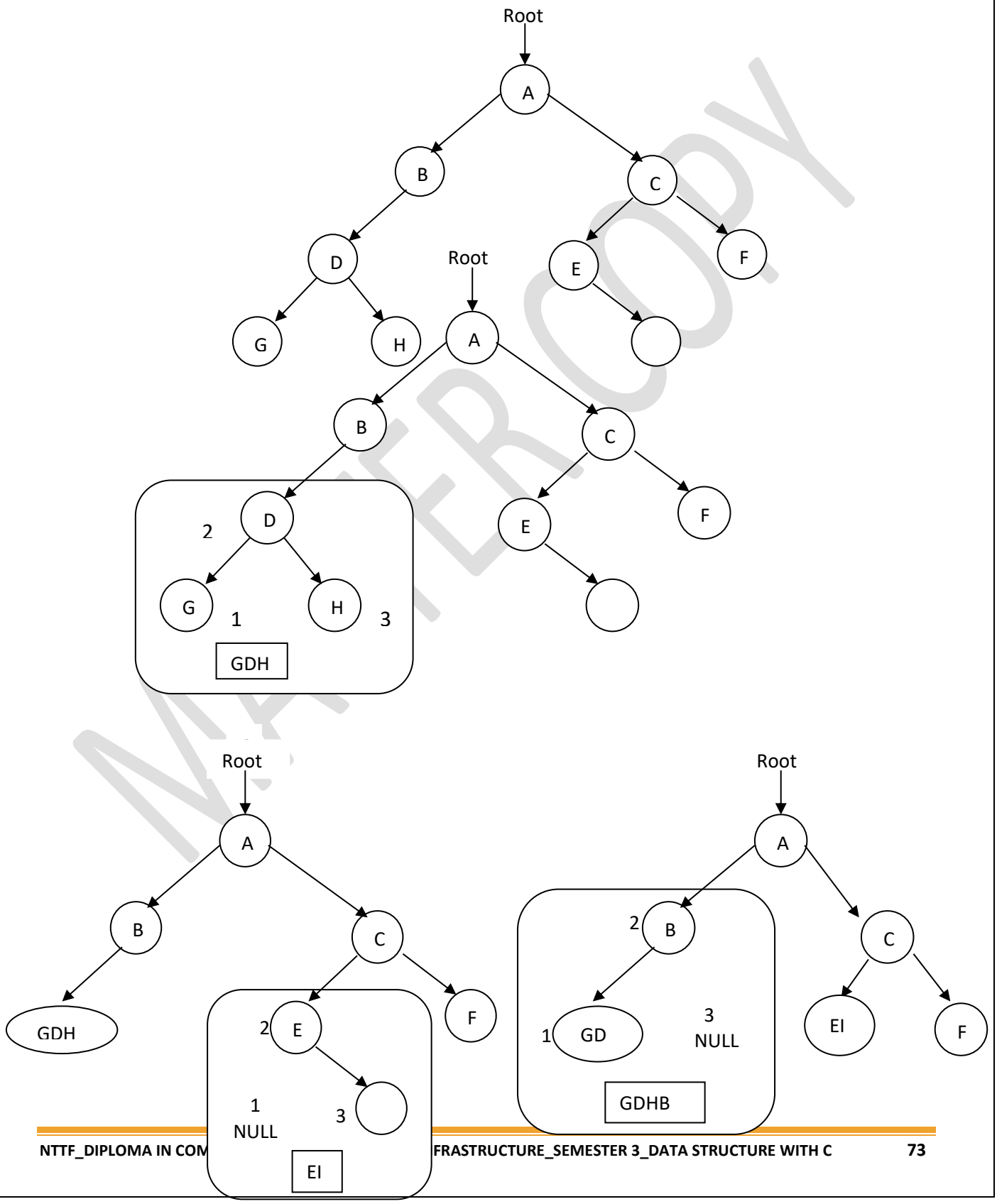

Root
1
A
2
BDG
3
CEIF


Root
ABDGHCEI
F

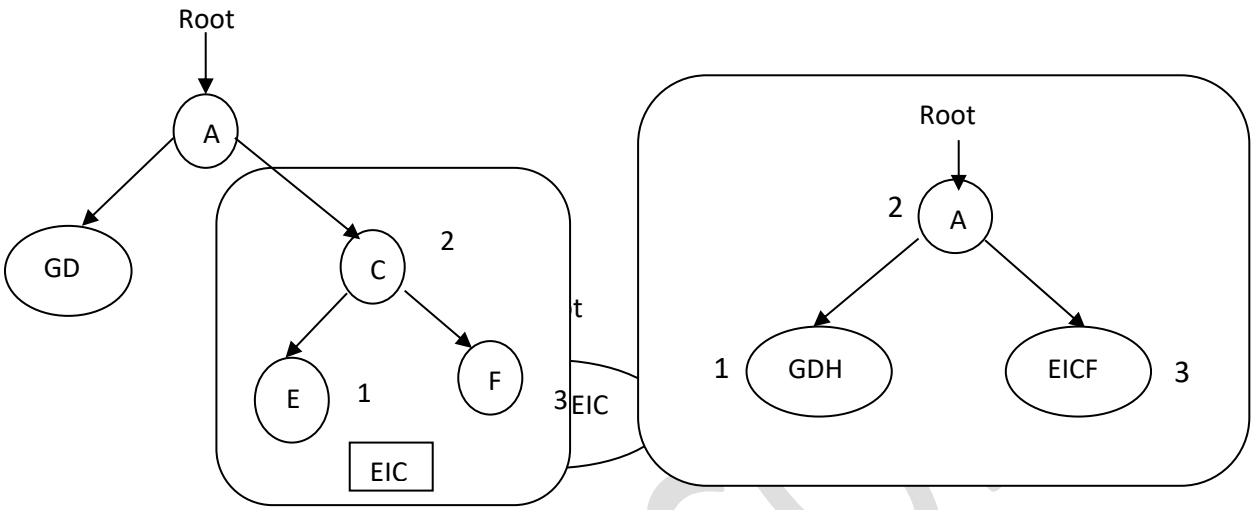
MASTER COPY

Derived preorder traversal of above tree is - ABDGHCEIF

2. Example: Traversal of Inorder (Left, Root, Right)

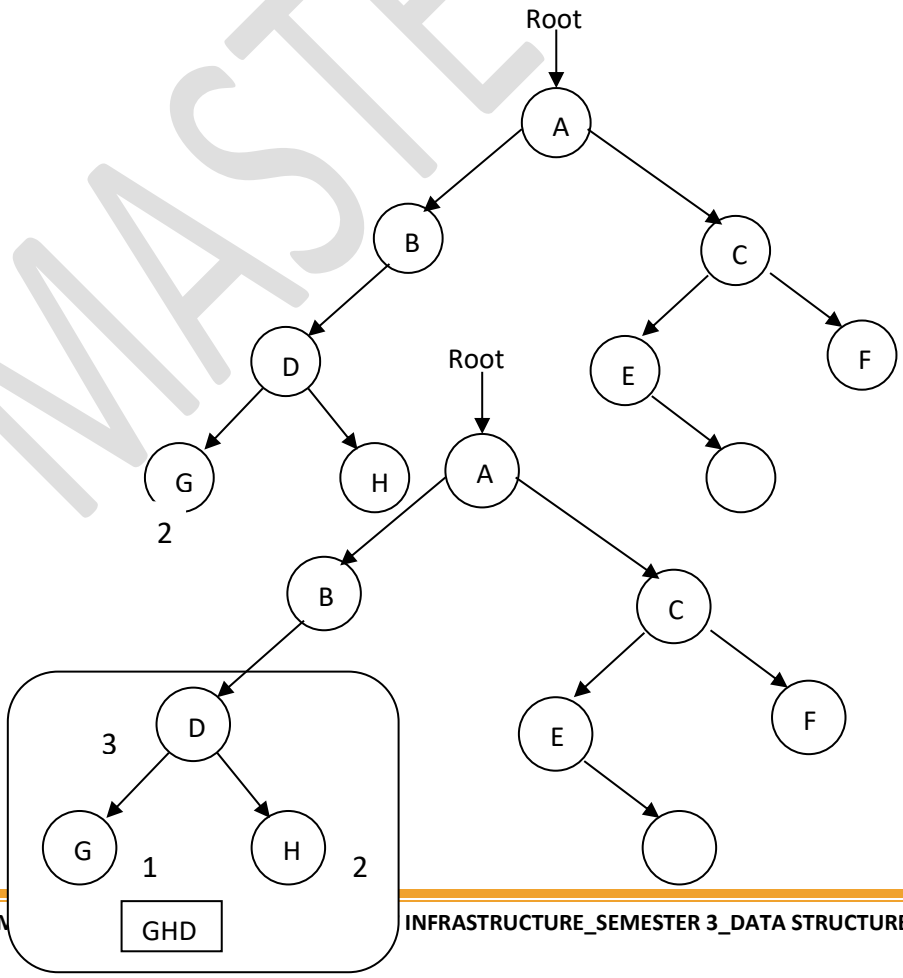


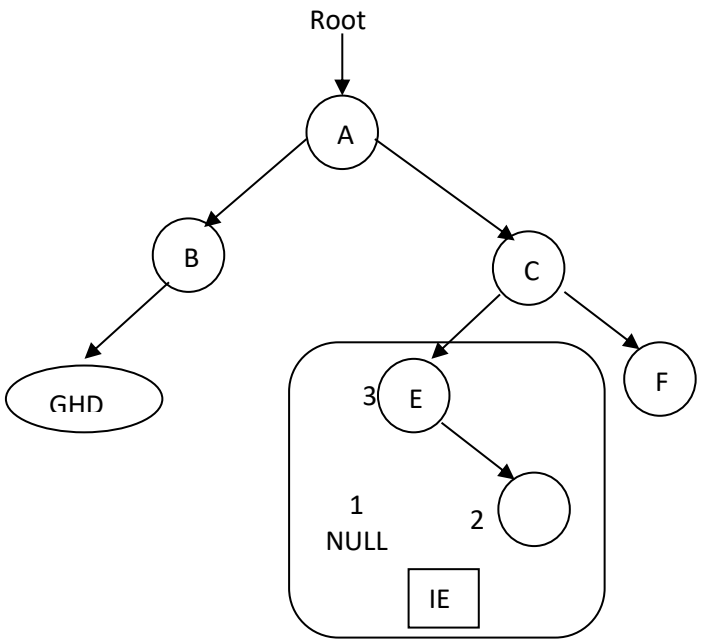
MASTERCOPY

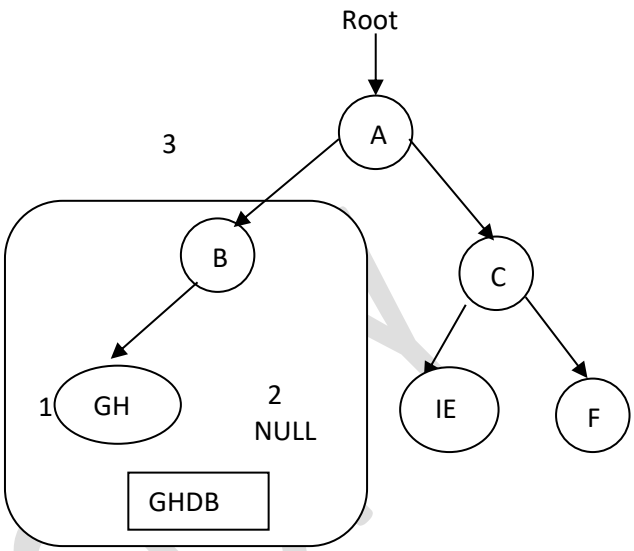


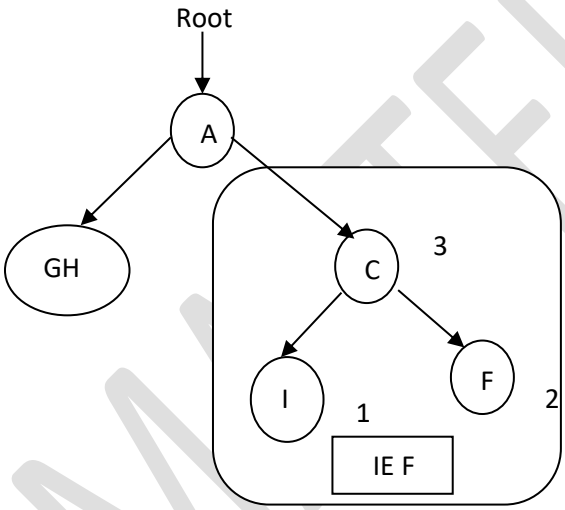
Derived Inorder traversal of above tree is - GDHBAEICF

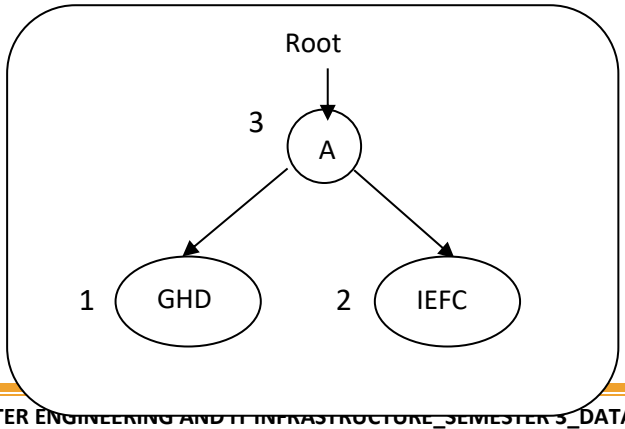
1. Example : Traversal of Postorder (Left, Right, Root,)





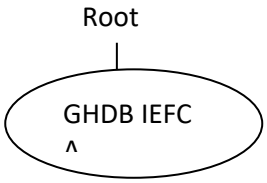






2

MASTER COPY



Derived Postorder traversal of above tree is - GHDBIEFCA

5. Tree Deletion

Its very important when it comes to the tree deletion, when a node is deleted from the tree, the ordering of the tree should be maintained. Even after deleting a node from a binary search tree, the elements in the left subtree should be lesser than its parent node and the elements in the right subtree should be greater than or equal to its parent node.

Whenever we delete a node from a tree the following cases must be checked before deleting a node from a binary search tree.

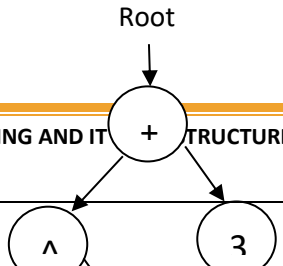
5. Expression Tree.

A sequence of operators and operands that reduces a single value is called an expression tree. A binary tree is said to an expression tree, then it must satisfy following properties.

- Any leaf is an operand
- Root and internal nodes are operator
- The subtrees represent the sub expressions with root of the sub tree as an operator.

Let's see how an arithmetic expression can be represented as an expression tree.

((6 + (3 -2) * 5) ^ 2 + 3)



MASTER COPY

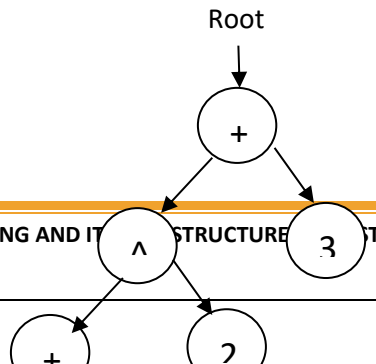
Fig (1)

Procedure to create an expression tree

- Create a postfix expression.
- Scan the symbols from left to right from the postfix expression
- Create a node for the scanned symbol
- If the symbol is an operand push the corresponding node into the stack.
- If the symbol is an operator, pop one node from the stack and attach to the right of the corresponding node with an operator. The first popped node represents the right operand. Pop one more node from stack and attach to the left of the operator node. Push the corresponding operator node on the stack again.
- Repeat the same operation for all the symbols scanned from the input expression. Finally scanning of all the symbols from the postfix expression is over, address of the root node of the expression tree is on top of the stack.

Note

- *An expression tree constructed and perform preorder traversal, then we will get prefix expression*
- *An expression tree constructed and performs Post order traversal, then we will get postfix expression*
- *An expression tree with In order traversal, then we will get infix expression*



MASTER COPY

Preorder Traversal	Inorder Traversal	Postorder Traversal
+^+6*-32523	((6+(3-2)*5)^2+3)	632-5*+2^3+

5.4 Applications of Trees

1. Searching and Sorting
2. Manipulation of arithmetic expression
3. Construction of Symbol table
4. Syntax analysis of Compiler Design.

MASTER COPY

Chapter – 6 SORTING

6.1 Introduction, Bubble Sort

Sorting is an important concept that is extensively used in the field of computer science and day today life. *Sorting is the process of arranging elements in a specific order*. Locating elements from the sorted list is very easy than from an unsorted list.
Sorting is a process of rearranging elements to a specific order. It can ascending order, descending order or alphabetical order.
Scenario

Suppose searching for a telephone number of a person in an unsorted telephone directory is a tedious job than searching for the same in a sorted telephone directory.
Its often prefer to arrange applications, list, elements, data in a sorted order for better convenience. More often programmers will be working with large amount of data and it may be necessary to arrange them in either ascending or descending order. '*The process of rearranging the given elements in ascending or descending order is called as sorting*'

Various Sorting Techniques

1. Bubble Sort
2. Selection Sort
3. Merge Sort
4. Quick Sort (Partition Exchange Sort or Divide & Conquer)
5. Radix Sort
6. Insertion Sort (Simple Insertion, Shell Sort, Address Calculation Sort)
7. Tree Sorts
   - 7.1 Binary Tree Sort
   - 7.2 Heap Sort
     - 7.2.1 Ascending Heap
     - 7.2.2 Descending Heap

MASTER COPY

Bubble Sort

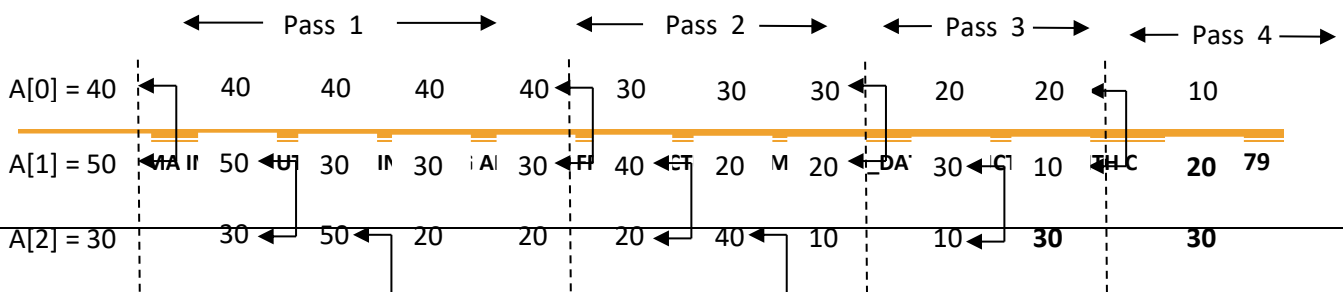


Fig 6.1

Bubble Sort is also called as *Sinking Sort*, in which higher weighted elements sinks down to the bottom or light weight elements floats at top of list. In the above example an array of five elements, in bubble sort comparison happens between two elements from first to last and higher elements move to last index and smaller elements moves to initial indexes

Advantages of Bubble Sort

- Very simple and easy to program
- Straight forward approach
- Reasonable to use for small set of elements to sort

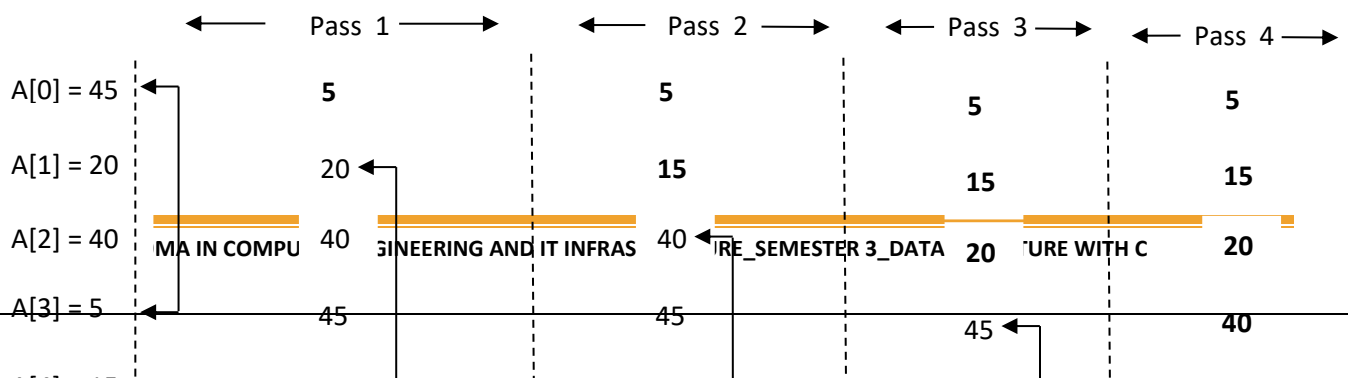
Disadvantages of Bubble Sort

- Its run's slowly when the elements are huge hence it is inefficient
- Though it sort the elements, it does n-1 passes to sort entire list
- Large number of comparisons happens when this algorithm is used

Selection Sort

As the name indicates, we find the smallest item from the list of array and interchange it with first item. Obtain the second smallest item from the list and interchange it with second position of array. Continue the same sequence till all elements are compared. Finally all the elements will be arranged in ascending order.

This technique *selects* the '*smallest item'* from the list and finds position for it, hence it is called as *selection sort*



MASTER COPY

Fig 6.2

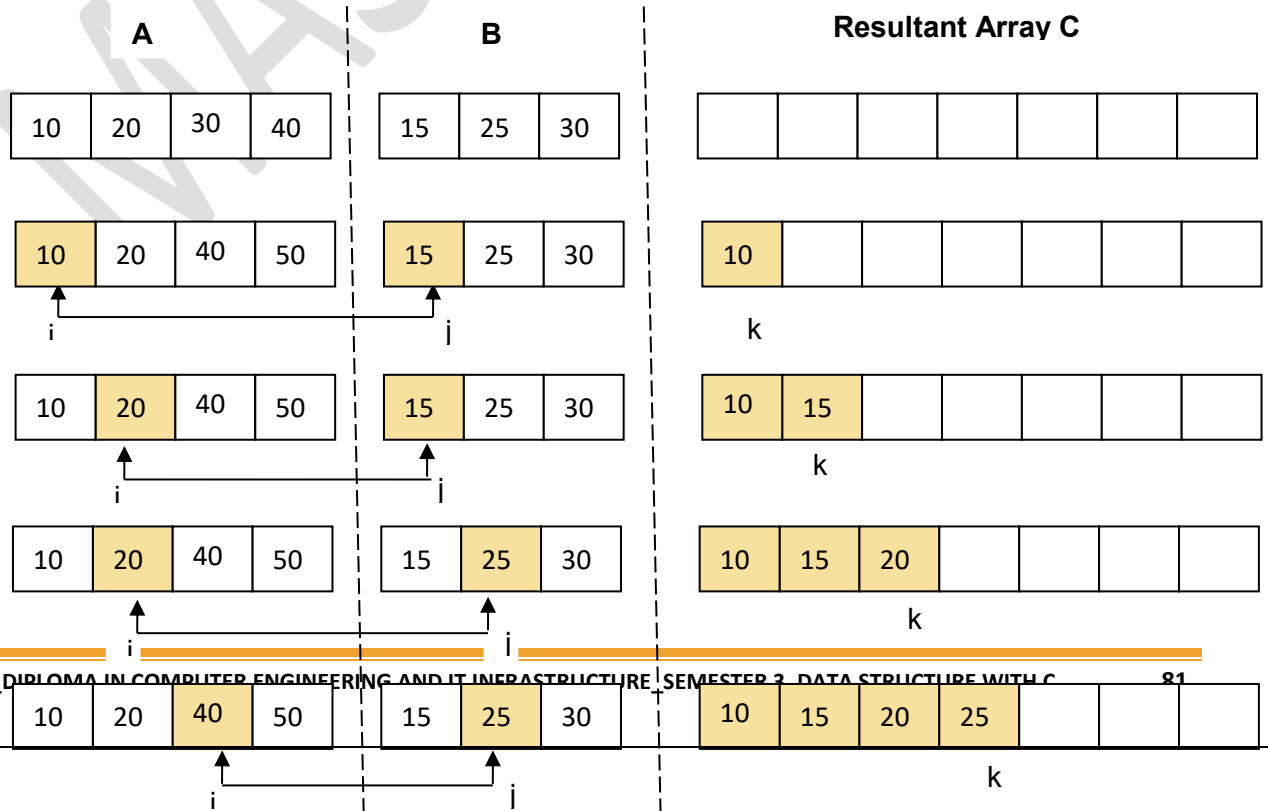
Fig 6.2 shows the working of 'Selection Sort'. Select the minimum from list a[3] ie 5 will be replaced with the first index a[0] and a[0]th Value 45 will be placed on a[3] position. Similarly in 2nd pass next minimum in the list which is a[4] ie number 15 will be swapped with a[1] index. This process continued till all the elements are got sorted.

6.2 Merge Sort

Merge sort is a sorting technique which is used to sort two sorted arrays. It is important that it will work only if the two input arrays are already sorted. Merge sort preferably used to merge two arrays and sort the content in it.

Working of Merge Sort

In Merge sort, the two input arrays are already sorted. The first element from the first array will be compared with the first element in the second array; the smallest from the list is selected to put in the resultant array.



MASTER COPY

According to merge sort algorithm, first array(A) first element(10) will be k

point by i (index 0) and first element(15) of second array(B) point by j(index 0). The values of i and j pointing indexes(10,15) are compared by the algorithm and minimum(10) form either first array or second array is send to the resultant array(C) k pointing index(0).

If the element added into resultant array from A array then i will be incremented, if the element from B array added into resultant array then j will incremented.

The algorithm continue comparing elements of new i and j pointing values and swap. Finally you could see the resultant array is been sorted.

6.3 Radix Sort

This is the method of sorting is used in card sorting machines, Here we assume that all the numbers to be sorted are of equal digits. There are 10 pockets created ranging from 0 to 9. Initially all the pockets are empty. Select the numbers one by one and separate the least significant digits. The separated digit belongs to the number ranging from 0 to 9, and then insert the scanned number into the appropriate pocket ranging from 0 to 9. If the pocket is non empty add the number at the rear end of the pocket. Generally the pockets are created with linked list.

Say for example 46 is the first number in the list then the least significant digit of 46 is 6, then 46 must insert into 6th pocket. Continue the same operation on all the elements in the list.

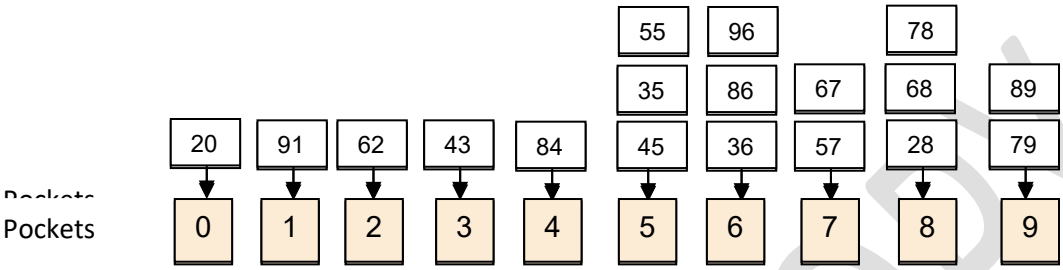
Consider the following numbers; let's see how merge sort is doing the sort on these numbers

57, 45, 36, 91, 28, 79, 35, 68, 89, 20, 62, 43, 84, 55, 86, 96, 78, 67

MASTER COPY

57, 45, 36, 91, 28, 79, 35, 68, 89, 20, 62, 43, 84, 55, 86, 96, 78, 67

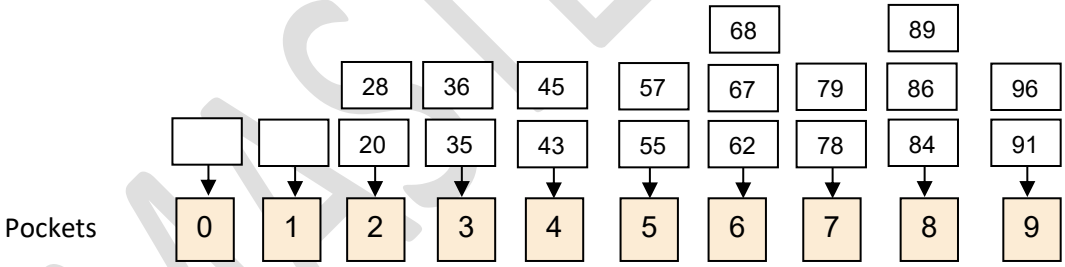
In first pass, all these input numbers will be inserted to certain pockets on the basis of least significant digit of the number, 57 goes to 7th pocket and 45 goes to 5th pocket etc so on.



Take out the elements from pocket 0 to 9 in same order ranging from first to last. Then the elements order will be the following

20,91,62,43,84,45,35,55,36,86,96,57,67,28,68,78,79,89.

In second pass, empty all the pockets, take the numbers one by one and separate 2nd least significant digit and insert into the appropriate pockets ranging from 0 to 9.



Take out the elements from pocket 0 to 9 in same order ranging from first to last. Then the elements order will be the following

20,28,35,36,43,45,55,57,62,67,68,78,79,84,86,89,91,96

If you look at these numbers, you can come to know that the elements are sorted in ascending order. This merge sort technique is quit applicable if the entire set of element to be sorted or equal in length (ie number of digits should be equal)

6.4 Quick Sort

Quick sort is one of the very efficient sorting techniques for a large set of

MASTER COPY

data. It works well for a large amount of data in an unsorted array.

Quick sort divide the entire list into two partitions based on a key element. The key element is also called as pivot element. The elements which are lesser than the key element will be in first half partition and the elements which are greater than the key element will be in second half partition.

The newly generated two partitions (first half and second half) will be send to quicksort() function to continue divide the partition based on the another key element. Continue the process of dividing partitions recursively till the maximum possible divisions can be created.

Once the algorithm completes the iteration you can see all the elements in the unsorted array will be get sorted.

6.5 Shell Sort

Shell sort is a highly efficient sorting algorithm and is based on insertion sort algorithm. This algorithm avoids large shifts as in case of insertion sort, if the smaller value is to the far right and has to be moved to the far left.

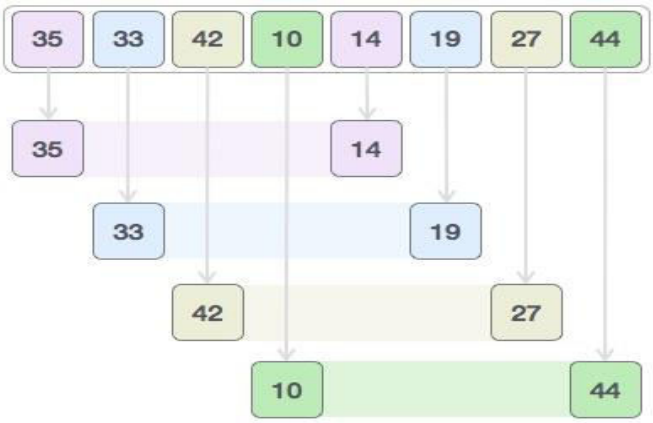
This algorithm uses insertion sort on a widely spread elements, first to sort them and then sorts the less widely spaced elements. This spacing is termed as interval. This interval is calculated based on Knuth's formula as

$h = h * 3 + 1$ where − h is interval with initial value 1

This algorithm is quite efficient for medium-sized data sets as its average and worst case complexity are of O(n), where n is the number of items.

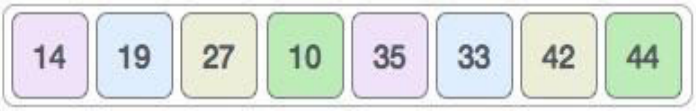
How Shell Sort Works?

Let us consider the following example to have an idea of how shell sort works. We take the same array we have used in our previous examples. For our example and ease of understanding, we take the interval of 4. Make a virtual sub-list of all values located at the interval of 4 positions. Here these values are {35, 14}, {33, 19}, {42, 27} and {10, 44}

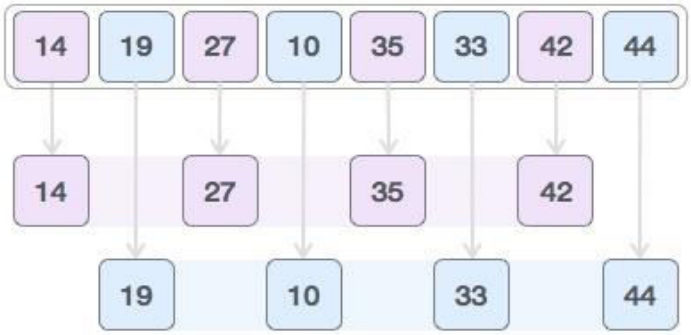


We compare values in each sub-list and swap them (if necessary) in the original array. After this step, the new array should look like this −

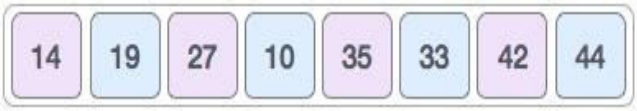
MASTER COPY



Then, we take interval of 2 and this gap generates two sub-lists - {14, 27, 35, 42}, {19, 10, 33, 44}



We compare and swap the values, if required, in the original array. After this step, the array should look like this



Finally, we sort the rest of the array using interval of value 1. Shell sort uses insertion sort to sort the array.
Following is the step-by-step depiction

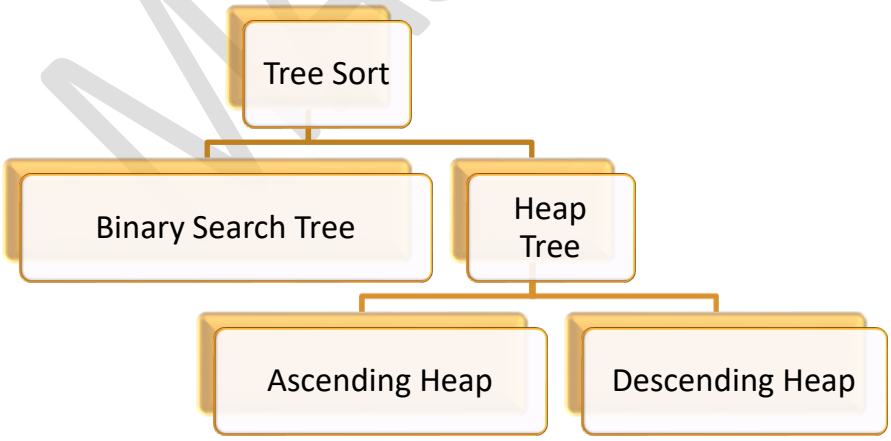
MASTER COPY

14	19	27	10	35	33	42	44
14	19	27	10	35	33	42	44
14	19	27	10	35	33	42	44
14	19	27	10	35	33	42	44
14	19	10	27	35	33	42	44
14	10	19	27	35	33	42	44
10	14	19	27	35	33	42	44
10	14	19	27	35	33	42	44
10	14	19	27	33	35	42	44
10	14	19	27	33	35	42	44

We see that it required only four swaps to sort the rest of the array.

Tree Sort

There are two kinds of tree sort we can perform



Binary Search Tree (BST)

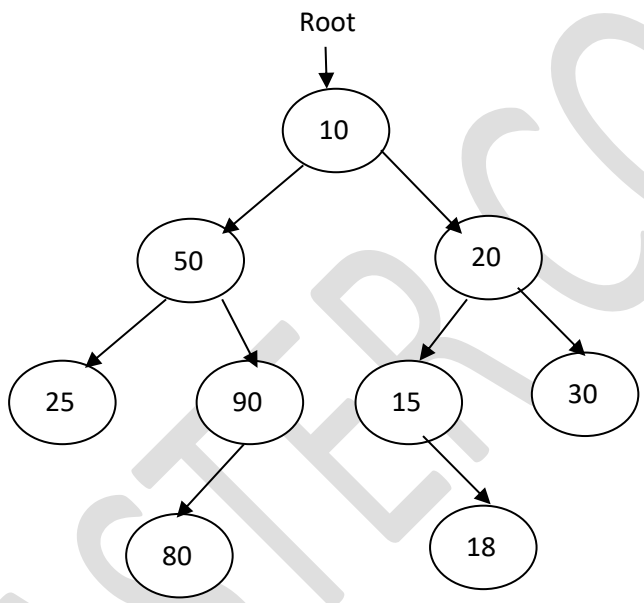
A **binary search tree** is a binary tree in which every node say x in the tree, elements in the left sub tree are less than info(x). And elements in the right sub tree are greater than equal to info(x). Every node in tree should satisfy this condition if left sub tree or right sub tree exists.

If a binary tree is said to be a Binary Search Tree, then every node in a tree should satisfy following condition.

1. The left sub tree node value must be less than its parent's node.
2. The right sub tree node value must be greater than or equal to its parents node.
3. If every node in a tree must satisfy this condition then the tree is said to be binary search tree.

Note : BST Already discussed in Tree type in 6th Unit

Create a Binary Search Tree and perform Inorder traversal then we will get sorted tree.



6.6 Heap Sort

A Heap tree is a Complete Binary Tree where all the levels are full except possibly the last level i.e. if a tree exists with 'n' level then the tree is complete up to $n-1^{th}$ level so that the number of nodes at $n-1^{th}$ level is 2^{n-1}. And in N^{th} level if number of nodes is less than 2^n , then the nodes in that level should be completely filled only from left to right.

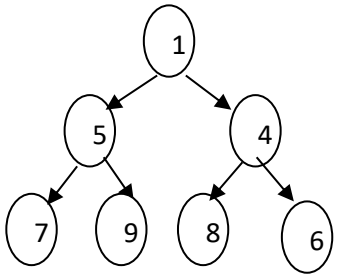
In other words a tree with n level exists, till last previous levels the tree should be a complete binary tree ($n-1^{th}$ level 2^{n-1} node exists) and in last level (n^{th} level) nodes should fill from left to right.

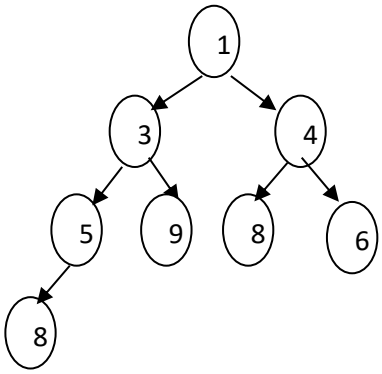
There are two kinds of heap can be created *Ascending heap* and *descending heap.*

Ascending Heap

MASTERCOPY

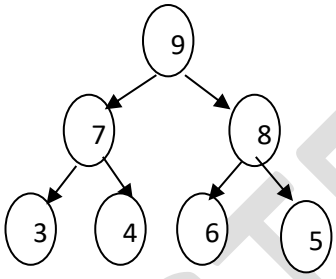
In ascending heap the root node contains the least element and each node in a tree is lesser than its left child and right child. Every node in a tree satisfies this condition then it's a ascending heap.

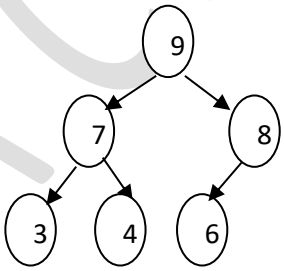




Descending Heap

In ascending heap the root node contains the largest element and each node in a tree is greater than its left child and right child. Every node in a tree satisfies this condition then it's a descending heap.





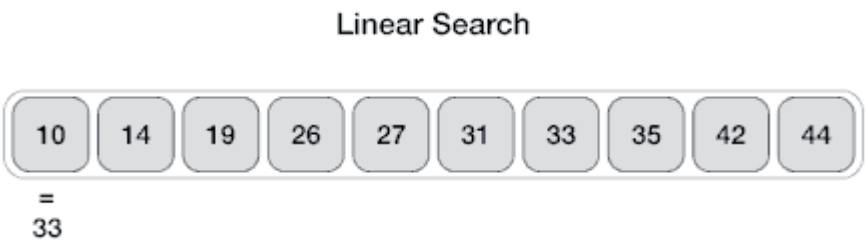
MASTER COPY

Chapter – 7 SEARCHING

Searching is the process of finding a key element of key item in a list of elements.

7.1 Basic search techniques, algorithmic notation

Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.



7.2 Searching, searching an ordered Table

When data items are stored in a collection such as a list, we say that they have a linear or sequential relationship. Each data item is stored in a position relative to the others. In Python lists, these relative positions are the index values of the individual items. Since these index values are ordered, it is possible for us to visit them in sequence. This process gives rise to our first searching technique, the **sequential search**.

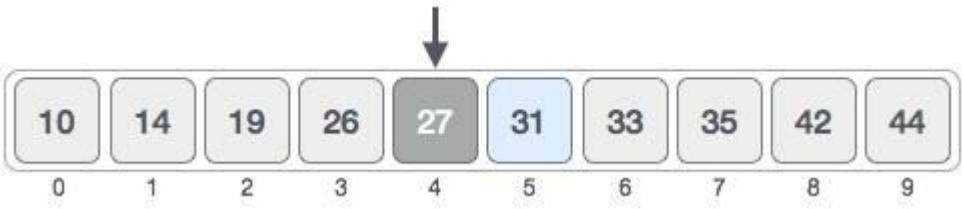
Starting at the first item in the list, we simply move from item to item, following the underlying sequential ordering until we either find what we are looking for or run out of items. If we run out of items, we have discovered that the item we were searching for was not present.

7.3 Binary Search

Binary search is a fast search algorithm with run-time complexity of O(log n). This search algorithm works on the principle of divide and conquers. For this algorithm to work properly, the data collection should be in the sorted form.

Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the sub array reduces to zero.

How Binary Search Works?



How Binary Search Works?

Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

We change our low to mid + 1 and find the new mid value again.

low = mid + 1
mid = low + (high - low) / 2

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Interpolation Search

This technique also works only if the items in the array are arranged in ascending or descending order. This technique is better than binary search when the items are uniformly distributed between first index to the last index of an array.

The interpolation search technique is almost similar to binary search except mid index calculation formula.

In binary search mid value can be calculated using

Mid = (low + high) /2

Where as in interpolation search, mid is calculated using the following formula

Mid = low+ (high –low) * ((key- a[low])/ (a[high]-a[low]));

7.4 Tree searching: Binary search

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties −

The left sub-tree of a node has a key less than or equal to its parent node's key.

The right sub-tree of a node has a key greater than to its parent node's key.

Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right sub-tree and can be defined as −

left_subtree (keys) ≤ node (key) ≤ right_subtree (keys)

MASTER COPY

BST is a collection of nodes arranged in a way where they maintain BST properties. Each node has a key and an associated value. While searching, the desired key is compared to the keys in BST and if found, the associated value is retrieved.

Search Operation

Whenever an element is to be searched, start searching from the root node. Then if the data is less than the key value, search for the element in the left subtree. Otherwise, search for the element in the right subtree. Follow the same algorithm for each node.

```
struct node* search(int data){
struct node *current = root;
   printf("Visiting elements: ");

while(current->data != data){

if(current != NULL){
        printf("%d ",current->data);

//go to left tree
if(current->data > data){
           current = current->leftChild;
}//else go to right tree
else{
           current = current->rightChild;
}

//not found
if(current == NULL){
return NULL;
}
}
}
return current;
}
```

MASTER COPY